

Transformers: Fundamentos y Aplicación en LLMs



Víctor Sierra Vicén

Director: Ricardo López Ruiz

Trabajo de fin de grado de Matemáticas

Universidad de Zaragoza

Julio de 2026

Resumen

Actualmente, la Inteligencia Artificial (IA) ha dejado de ser una promesa futurista para convertirse en una realidad omnipresente. Nos encontramos en una etapa de evolución tecnológica constante donde la automatización y el análisis de datos están transformando sectores críticos de la sociedad como la sanidad, la administración pública y la industria del entretenimiento.

La motivación principal de este trabajo nace del reciente cambio de paradigma introducido por las arquitecturas basadas en Transformers. Estos modelos han revolucionado el campo del Procesamiento del Lenguaje Natural (PLN), permitiendo la creación de asistentes conversacionales (*chatbots*) capaces de mantener una coherencia contextual y semántica.

El presente proyecto tiene un doble objetivo: profundizar en la formalización matemática de la arquitectura Transformer y aplicar dichos conceptos en la implementación de un chatbot capaz de responder preguntas sobre el propio modelo, cerrando así el vínculo entre la teoría desarrollada y su aplicación práctica.

En el primer capítulo se introducen los conceptos fundamentales que sustentan el trabajo: el *Machine Learning*, las redes neuronales, el *Deep Learning* y el problema del desvanecimiento del gradiente, el cual servirá de hilo conductor a lo largo de todo el documento.

El segundo capítulo constituye el núcleo teórico del trabajo. En él se desarrolla en profundidad la arquitectura del Transformer: el mecanismo de autoatención y su formalización, la codificación posicional, y el análisis matemático de la estabilidad del gradiente mediante conexiones residuales y normalización de capa. El capítulo concluye con una descripción del modelo GPT como especialización del Transformer para la generación de lenguaje.

Finalmente, en el tercer capítulo se presenta una aplicación práctica: la construcción de un chatbot especializado en el modelo Transformer, obtenido mediante el ajuste fino de un modelo GPT preentrenado en español. Las consecuencias del tamaño y la diversidad del conjunto de datos (*dataset*) en la capacidad de generalización del modelo son analizadas comparando cuatro versiones del dataset.

Complementariamente, en los anexos se incluye el análisis de las arquitecturas RNN y CNN que precedieron al Transformer, el historial del entrenamiento del chatbot, un estudio de la reproducibilidad de las respuestas generadas por este y el código fuente empleado.

Abstract

Artificial Intelligence (AI) has evolved from a futuristic promise into a pervasive reality. We are immersed in an era of constant technological progress where automation and data analysis are transforming critical sectors of society such as healthcare, public administration and the entertainment industry.

The main motivation of this work stems from the recent paradigm shift introduced by Transformer-based architectures. These models have revolutionized the field of Natural Language Processing (NLP), enabling the creation of conversational assistants (*chatbots*) capable of maintaining contextual and semantic coherence.

This project has two objectives: to delve into the mathematical formalization of the Transformer architecture and to apply these concepts in the implementation of a chatbot capable of answering questions about the model itself, thereby closing the link between the theory developed and its practical application.

The first chapter introduces the fundamental concepts underpinning the work: Machine Learning, neural networks, Deep Learning and the vanishing gradient problem, which serves as a guiding thread throughout the entire document.

The second chapter constitutes the theoretical core of the work. It develops the Transformer architecture in depth: the self-attention mechanism and its formalization, positional encoding, and the mathematical analysis of gradient stability through residual connections and layer normalization. The chapter concludes with a description of the GPT model as a specialization of the Transformer for language generation.

Finally, the third chapter presents a practical application: the construction of a chatbot specialized in the Transformer model, obtained by fine-tuning a GPT model pre-trained in Spanish. The effect of dataset size and diversity on the model's generalization capacity is analyzed by comparing four dataset versions.

Additionally, the appendices include the analysis of the RNN and CNN architectures that preceded the Transformer, the training history of the chatbot, a reproducibility study of the generated responses, and the source code employed.

Índice general

Resumen	III
Abstract	V
1. Introducción	1
1.1. Machine Learning	1
1.1.1. Tipos de Aprendizaje	1
1.2. Redes Neuronales	1
1.2.1. Aprendizaje de una Red Neuronal	2
1.3. Deep Learning	3
1.3.1. La Necesidad de la Profundidad Arquitectónica	5
2. Modelo Transformer	7
2.1. Motivación	7
2.2. Estructura Encoder-Decoder	7
2.3. Atención: Mecanismo de Self-Attention	9
2.3.1. Atención de Producto Escalar Escalado	10
2.3.2. Atención Multi-Cabezal	12
2.3.3. Aplicaciones de la Atención en el Modelo	14
2.4. Codificación Posicional	15
2.5. Estabilidad del Gradiente en el Transformer	17
2.5.1. Conexiones Residuales	17
2.5.2. Normalización de Capa	19
2.5.3. Gradientes Bien Comportados en el Transformer	21
2.6. De Transformers a LLMs: el Modelo GPT	23
2.6.1. Objetivo de Entrenamiento	23
2.6.2. Preentrenamiento y Ajuste Fino	23
3. Construcción de un Chatbot sobre el Modelo Transformer	25
3.1. Planteamiento y Modelo Base	25
3.2. Dataset y Entrenamiento	25
3.3. Resultados	26
3.4. Conclusiones y Trabajo Futuro	27
Bibliografía	29
A. Modelos RNN - CNN	31
A.1. Redes Neuronales Recurrentes	31
A.2. Redes Neuronales Convolucionales	33
A.2.1. Estructura de las CNN	35
B. Función Softmax	39

C. Desarrollo y Evaluación del Chatbot	41
D. Reproducibilidad de las Respuestas	47
E. Código Fuente	51
F. Implementación del Chatbot	55

Capítulo 1

Introducción

1.1. Machine Learning

El **Machine Learning** es una rama de la Inteligencia Artificial que se centra en el desarrollo de sistemas que pueden aprender y mejorar su rendimiento por sí mismos, sin ser programados explícitamente. Estos sistemas son entrenados con un gran volumen de datos, lo que les permite identificar patrones y establecer relaciones que usan para tomar decisiones y realizar predicciones de manera autónoma.



Figura 1.1: Funcionamiento general del proceso de Machine Learning.

1.1.1. Tipos de Aprendizaje

Los algoritmos de Machine Learning se clasifican en tres categorías principales según la naturaleza de su retroalimentación durante el aprendizaje:

- **Aprendizaje Supervisado:** El modelo se entrena con un conjunto de datos etiquetado, es decir, pares de entrada y salida deseada (x, y) . El objetivo es aprender una función $f : X \rightarrow Y$ que minimice el error entre la predicción y la etiqueta real. Este es el enfoque utilizado en tareas de clasificación (etiquetas discretas) y regresión (valores continuos).
- **Aprendizaje No Supervisado:** El algoritmo recibe datos sin etiquetar y debe encontrar estructuras subyacentes o patrones por sí mismo. Es común en tareas de *clustering* (agrupamiento) o reducción de dimensionalidad.
- **Aprendizaje por Refuerzo:** El agente aprende a tomar decisiones secuenciales interactuando con un entorno. No recibe etiquetas explícitas, sino señales de recompensa o castigo en función de sus acciones, buscando maximizar la recompensa acumulada a largo plazo.

El principal desafío en cualquiera de estos enfoques reside en la capacidad de generalización. Un modelo útil no es aquel que simplemente memoriza los datos de entrenamiento (fenómeno conocido como *overfitting*), sino aquel que consigue abstraer las reglas subyacentes para realizar predicciones precisas sobre datos nuevos nunca antes vistos.

1.2. Redes Neuronales

Las **redes neuronales** son modelos computacionales inspirados en el cerebro biológico, las cuales están compuestas por nodos interconectados (neuronas artificiales) que procesan información. Estas neu-

ronas trabajan en conjunto para realizar tareas de procesamiento y análisis de datos, como clasificaciones o predicciones.

Los elementos que forman una neurona son los siguientes:

- **Entrada:** La entrada o *input*, que se puede representar como $\mathbf{x} = (x_1, \dots, x_n)^T$ con $x_i \in \mathbb{R}$, $i = 1, \dots, n$. Cada x_i corresponde a una señal de una neurona de la capa anterior, o en caso de ser de la capa de entrada, a una variable externa.
- **Pesos:** Cada una de las entradas está asociada a un parámetro peso, denotado por el vector $\mathbf{w} = (w_1, \dots, w_n)^T$, que determina su importancia.
- **Sesgo b :** Es un término independiente que permite desplazar el umbral de activación de la neurona, donde $b \in \mathbb{R}$.

El procesamiento interno de una neurona consta de dos etapas. En primer lugar, se calcula la combinación lineal de las entradas con sus respectivos pesos, sumando el sesgo:

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b \quad (1.1)$$

En segundo lugar, para que el modelo sea capaz de aprender representaciones que vayan más allá de simples relaciones lineales, a este resultado escalar z se le aplica una **función de activación** no lineal $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, produciendo la salida final y :

$$y = \sigma(z) = \sigma(\mathbf{w}^T \mathbf{x} + b) \quad (1.2)$$

Al agrupar múltiples neuronas interconectadas en capas, obtenemos una red neuronal de tipo *feed-forward* (hacia adelante). Si generalizamos la ecuación de la neurona individual a notación matricial, denotando por $\mathbf{a}^{(l-1)} \in \mathbb{R}^n$ el vector de activaciones (salidas) de la capa anterior, la salida de la capa actual l se calcula como:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}) \quad (1.3)$$

donde $\mathbf{W}^{(l)} \in \mathbb{R}^{m \times n}$ es la matriz que contiene todos los pesos que conectan la capa $l-1$ (de n neuronas) con la capa l (de m neuronas), y $\mathbf{b}^{(l)} \in \mathbb{R}^m$ es el vector de sesgos de dicha capa.

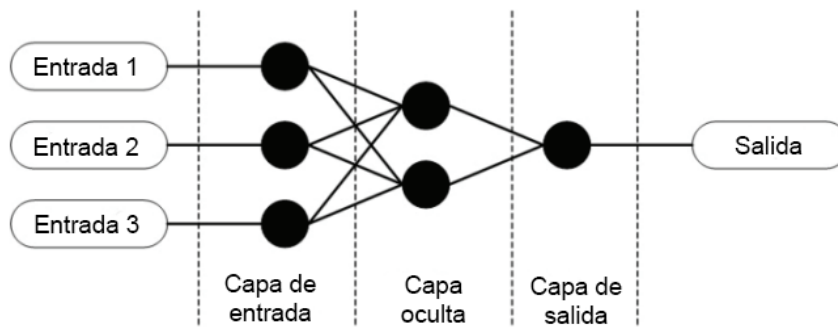


Figura 1.2: Esquema de una Red Neuronal

1.2.1. Aprendizaje de una Red Neuronal

Definición 1. Sea $\mathbf{y}$ el valor real e $\hat{\mathbf{y}}$ la predicción de la red. La **función de pérdida** es una función $\mathcal{L} : \mathbb{R}^p \times \mathbb{R}^p \rightarrow \mathbb{R}$ que mide la discrepancia entre ambas:

$$(\mathbf{y}, \hat{\mathbf{y}}) \mapsto \mathcal{L}(\mathbf{y}, \hat{\mathbf{y}})$$

El objetivo del aprendizaje es encontrar los parámetros (pesos y sesgos) que minimicen esta función. Para ello, se actualiza la matriz de pesos y el vector de sesgos de cada capa:

$$\begin{aligned} W^{(l)} &\leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \\ \mathbf{b}^{(l)} &\leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \end{aligned} \quad (1.4)$$

donde $\eta \in \mathbb{R}$ es la tasa de aprendizaje (*learning rate*), un escalar que controla a qué velocidad aprende la red neuronal. Si es muy pequeño, el aprendizaje se puede volver lento, pero si es muy grande el algoritmo puede volverse inestable y ser incapaz de estabilizarse en una solución óptima.

El **algoritmo de retropropagación** (*Backpropagation*, BP) [1] calcula estas derivadas parciales de forma eficiente aplicando repetidamente la regla de la cadena desde la capa de salida hacia las de entrada:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(l)}} \cdot \frac{\partial \mathbf{a}^{(l)}}{\partial \mathbf{z}^{(l)}} \cdot \frac{\partial \mathbf{z}^{(l)}}{\partial W^{(l)}} \quad (1.5)$$

Es mediante este proceso iterativo de propagación hacia adelante (inferencia) y retropropagación del gradiente (ajuste) a través del cual la red neuronal aprende a abstraer las características óptimas de los datos de entrenamiento.

Nota: La ecuación (1.5) ilustra la regla de la cadena a nivel conceptual. En la práctica, el cálculo exacto de estos gradientes implica multiplicaciones matriciales y productos de Hadamard, dependiendo de la función de activación escogida.

1.3. Deep Learning

El **Deep Learning** [2] es un campo del Machine Learning que utiliza redes neuronales con múltiples capas de procesamiento para modelar patrones complejos en los datos. Las primeras capas detectan detalles simples, y las capas sucesivas combinan esos detalles para reconocer formas cada vez más complejas y abstractas.

Definición 2. Una **red neuronal profunda** con M capas ($M \geq 2$) se define como la composición sucesiva de funciones seguidas de transformaciones no lineales. Dado un vector de entrada $\mathbf{x}$, la función global de la red $F : \mathbb{R}^n \rightarrow \mathbb{R}^p$ se expresa como:

$$F(\mathbf{x}) = (f^{(M)} \circ f^{(M-1)} \circ \dots \circ f^{(1)})(\mathbf{x}) \quad (1.6)$$

donde n es la dimensión de entrada y p la de salida. Cada función de capa m se define como:

$$f^{(m)}(\mathbf{a}^{(m-1)}) = \sigma(W^{(m)}\mathbf{a}^{(m-1)} + \mathbf{b}^{(m)}) = \mathbf{a}^{(m)}$$

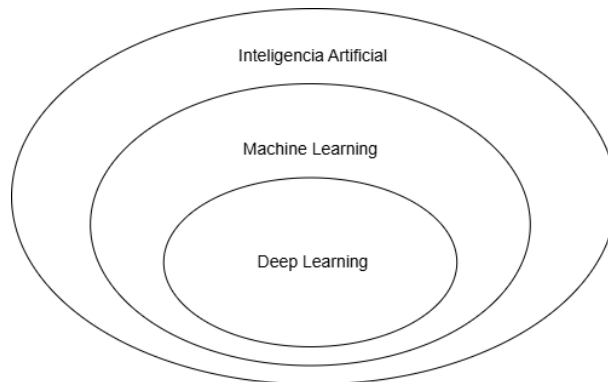


Figura 1.3: Esquema jerárquico de abstracción en Deep Learning.

Sin embargo, esta profundidad arquitectónica supone un desafío analítico severo. Al aplicar la regla de la cadena (1.5) para calcular el gradiente en las primeras capas de la red, nos vemos obligados a multiplicar una larga secuencia de matrices jacobianas:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(M)}} \cdot \frac{\partial \mathbf{a}^{(M)}}{\partial \mathbf{a}^{(M-1)}} \cdot \frac{\partial \mathbf{a}^{(M-1)}}{\partial \mathbf{a}^{(M-2)}} \cdots \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{a}^{(1)}} \cdot \frac{\partial \mathbf{a}^{(1)}}{\partial W^{(1)}} \quad (1.7)$$

es decir:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(M)}} \left(\prod_{m=2}^M \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right) \frac{\partial \mathbf{a}^{(1)}}{\partial W^{(1)}} \quad (1.8)$$

Observemos que la matriz jacobiana de cada capa, $\frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}}$, depende de los pesos de la red y de la derivada de la función de activación σ' , como hemos visto en la Ecuación (1.3).

Dado que algunas funciones de activación tradicionales como la sigmoide o la tangente hiperbólica tienen derivadas estrictamente acotadas por valores pequeños, las multiplicaciones sucesivas provocan una atenuación significativa en la norma del gradiente. Si asumimos que, como consecuencia de esto y de la inicialización de los pesos, la norma matricial de estas derivadas intermedias está acotada por un valor ω sistemáticamente menor que 1, cumple entonces que $\left\| \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right\| \leq \omega < 1$. Aplicando la propiedad multiplicativa de la norma, podemos acotar el producto total de las capas:

$$\left\| \prod_{m=2}^M \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right\| \leq \prod_{m=2}^M \left\| \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right\| \leq \omega^{M-1} \quad (1.9)$$

Dado que $\omega < 1$, si evaluamos el límite conforme la profundidad de la red M crece, el término ω decae hacia cero, lo que implica que el gradiente se anula:

$$\lim_{M \rightarrow \infty} \omega^{M-1} = 0 \implies \lim_{M \rightarrow \infty} \frac{\partial \mathcal{L}}{\partial W^{(1)}} = \mathbf{0} \quad (1.10)$$

Este fenómeno, conocido algebraicamente como el **desvanecimiento del gradiente** (*vanishing gradient*) [3], provoca que las primeras capas de la red dejen de aprender, ya que las actualizaciones de sus pesos ($W^{(1)}$) se vuelven computacionalmente nulas.

Como se puede apreciar en el mapa de calor de la Figura 1.4, la magnitud del gradiente en las dos últimas capas (la 9 y la 10) no baja del 0,4. Sin embargo, es al retroceder a partir de la capa 8 cuando esta disminuye drásticamente, por lo que como ya se ha mencionado, estas capas dejarán de aprender.

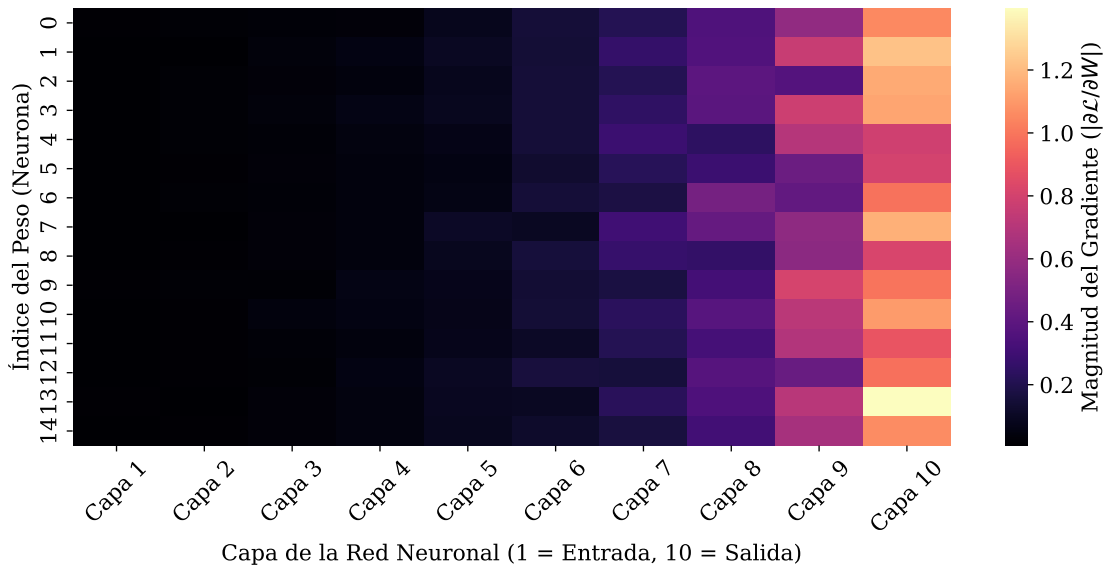


Figura 1.4: Mapa de calor simulado que ilustra el desvanecimiento del gradiente, generado con $\omega = 0,6$.

1.3.1. La Necesidad de la Profundidad Arquitectónica

Llegados a este punto, resulta natural plantearse una cuestión arquitectónica fundamental: si el incremento en el número de capas M es el causante directo del desvanecimiento del gradiente, ¿por qué no evitar el problema limitando el diseño a redes neuronales poco profundas?

Para poder responder a esta pregunta, primero veamos el siguiente Teorema:

Teorema 1.1 (Teorema de Aproximación Universal, Cybenko (1989)). *Sea $C(X, \mathbb{R})$ el espacio de las funciones continuas definidas sobre un subconjunto compacto $X \subset \mathbb{R}^n$. Sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación continua, acotada y no constante. Entonces, para cualquier función objetivo $f \in C(X, \mathbb{R})$ y para cualquier margen de error $\varepsilon > 0$, existe un número entero N (neuronas en la capa oculta) y un conjunto de parámetros reales (pesos $\mathbf{w}_i$, sesgos b_i y coeficientes v_i) tales que la función generada por la red:*

$$F(\mathbf{x}) = \sum_{i=1}^N v_i \sigma(\mathbf{w}_i^T \mathbf{x} + b_i) \quad (1.11)$$

cumple de forma estricta que $|F(\mathbf{x}) - f(\mathbf{x})| < \varepsilon$ para todo $\mathbf{x} \in X$.

Demostración. La demostración original, que emplea herramientas de Análisis Funcional (en particular el Teorema de Representación de Riesz y el Teorema de Hahn-Banach), puede consultarse en [4]. $\square$

Este teorema garantiza la existencia teórica de una red neuronal con una única capa oculta capaz de aproximar cualquier relación compleja en los datos, siempre que dicha capa sea suficientemente ancha ($N \rightarrow \infty$). Sin embargo, no se garantiza que la red sea eficiente o que pueda ser entrenada en la práctica. Para igualar el poder predictivo de una red profunda, una red con una única capa oculta requeriría un número de neuronas que crece de forma exponencial con respecto a la complejidad del problema, volviéndose computacionalmente intratable y extremadamente propensa al *overfitting*. Por ello, en la práctica se opta por apilar múltiples capas de menor tamaño, lo que permite una extracción jerárquica de características mucho más eficiente.

Es precisamente para superar estas limitaciones analíticas por lo que surge la necesidad de crear arquitecturas con un flujo de gradientes más robusto, como los Transformers.

Capítulo 2

Modelo Transformer

2.1. Motivación

En el Capítulo 1 se ha visto que el desvanecimiento del gradiente constituye el principal obstáculo para entrenar redes neuronales profundas (Ecuación 1.10). Este fenómeno se manifiesta igualmente en las dos arquitecturas que han dominado históricamente el Procesamiento del Lenguaje Natural: las Redes Neuronales Recurrentes (RNN) y las Redes Neuronales Convolucionales (CNN), cuyo análisis detallado se desarrolla en el Apéndice A.

El Transformer, introducido por Vaswani et al. [5] en 2017, aborda este problema mediante un cambio de paradigma radical: elimina por completo la recurrencia y la convolución, sustituyéndolas por un mecanismo de autoatención (*Self-Attention*) que permite a cualquier par de posiciones de la secuencia interactuar directamente, con independencia de la distancia que las separe. La longitud del camino entre dos tokens cualesquiera se reduce así a un único paso ($\mathcal{O}(1)$).

Junto a este mecanismo central, la arquitectura incorpora dos elementos que resuelven el problema del desvanecimiento del gradiente:

- **Conexiones residuales** [6]: introducen un camino directo para el flujo del gradiente que garantiza que la señal no se atenúe al retropropagarse a través de las capas.
- **Normalización de capa** (*Layer Normalization*) [7]: estabiliza la escala de las activaciones en cada sub-cap, manteniendo los gradientes estables independientemente de la profundidad de la red.

A lo largo de este capítulo se desarrolla en profundidad cada uno de estos elementos. En primer lugar, se describe la estructura codificador-decodificador del Transformer (Sección 2.2). A continuación, se formaliza el mecanismo de autoatención y su extensión multi-cabezal (Sección 2.3), seguido de la codificación posicional que aporta información del orden de la secuencia (Sección 2.4). Posteriormente, se analiza formalmente la estabilidad del gradiente que proporcionan las conexiones residuales y la normalización de capa (Sección 2.5). El capítulo concluye con la descripción del modelo GPT como especialización del Transformer para la generación de lenguaje (Sección 2.6).

2.2. Estructura Encoder-Decoder

La mayoría de los modelos neuronales de alto rendimiento emplean una estructura *Encoder-Decoder* (Codificador-Decodificador). En ella, el codificador convierte una secuencia de entrada $\mathbf{x} = (x_1, \dots, x_n)$ en una secuencia de representaciones continuas $\mathbf{z} = (z_1, \dots, z_n)$, procesando todas las posiciones de forma simultánea. Dado $\mathbf{z}$, el decodificador genera una secuencia de salida $\mathbf{y} = (y_1, \dots, y_m)$ elemento por elemento de forma auto-regresiva: en cada paso tiene acceso a la representación completa $\mathbf{z}$ del codificador y a los símbolos que ha generado previamente para producir el siguiente.

El Transformer adopta esta arquitectura general utilizando, tanto para el codificador como para el decodificador, capas de autoatención (*Self-Attention*) apiladas junto con redes *feed-forward* que se aplican

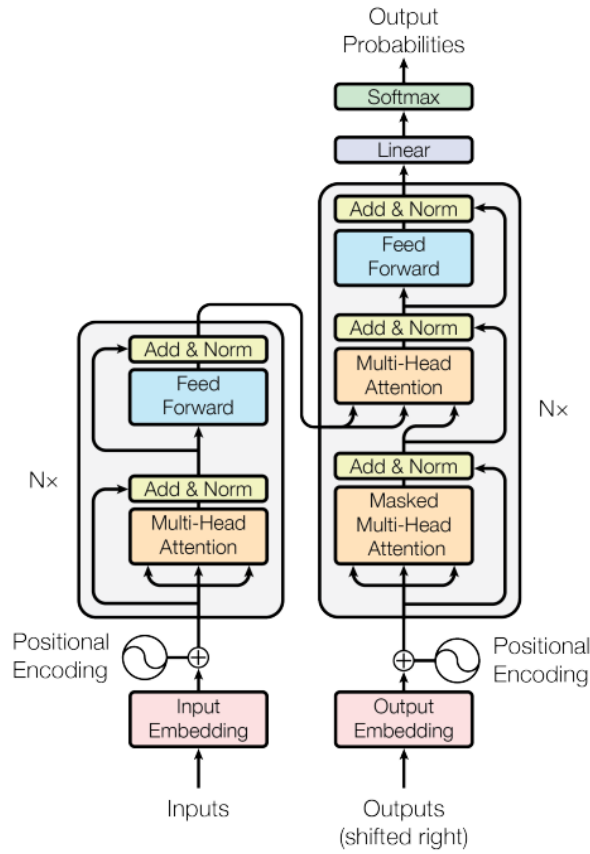


Figura 2.1: Arquitectura del modelo Transformer. Izquierda: codificador. Derecha: decodificador.

de forma independiente a cada posición de la secuencia. La Figura 2.1 muestra la arquitectura completa, publicada por primera vez en *Attention is all you need* [5].

- **Codificador:** Está compuesto por una pila de $L = 6$ bloques idénticos cuya estructura interna se muestra en la Figura 2.2. Cada bloque posee dos sub-capas:

1. Una capa de **autoatención multi-cabezal** (Sección 2.3.2).
2. Una única **red feed-forward** totalmente conectada, como las presentadas en la Sección 1.2.

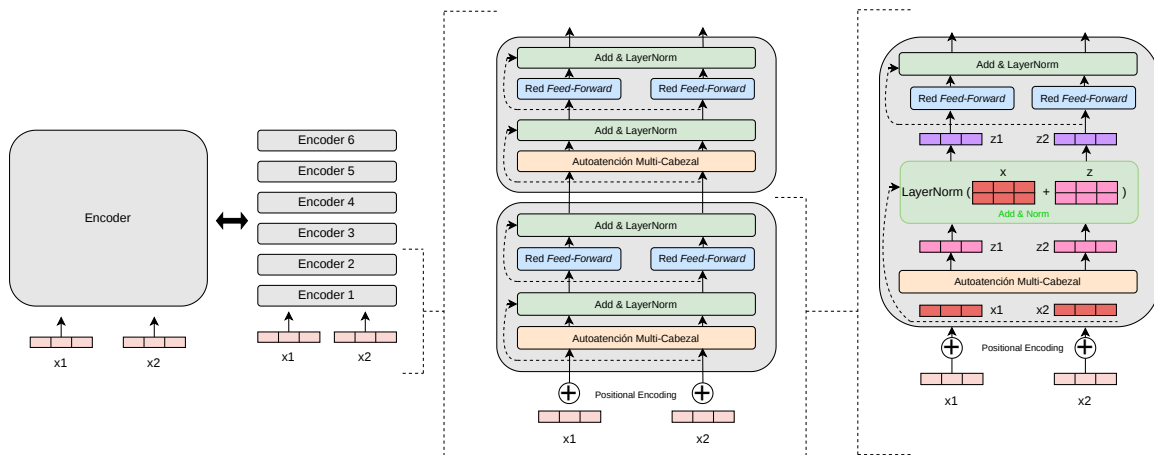


Figura 2.2: Estructura interna del codificador.

Para facilitar el flujo de gradientes en una red tan profunda, el diseño incorpora conexiones residuales alrededor de cada sub-capa, además de una normalización de capa tanto tras la capa de autoatención como tras la red *feed-forward*.

- **Decodificador:** Lo componen también $L = 6$ bloques idénticos. Cada una de ellas posee tres sub-capas, como se muestra en la Figura 2.3. Estas sub-capas emplean tres tipos de vectores, consultas (Q), claves (K) y valores (V), cuya definición formal se desarrolla en la Sección 2.3:
 1. Una capa de **autoatención multi-cabezal enmascarada**, que incorpora la máscara definida en la Ecuación (2.7) para preservar la propiedad auto-regresiva del modelo, impidiendo que la predicción en la posición i dependa de posiciones futuras.
 2. Una capa de **atención cruzada** (*Cross-Attention*), en la que las consultas Q provienen de la capa anterior del decodificador, mientras que las claves K y valores V provienen de la salida del codificador. Esto permite que cada posición del decodificador atienda a todas las posiciones de la secuencia de entrada.
 3. Una única **red feed-forward** totalmente conectada, idéntica a la del codificador.

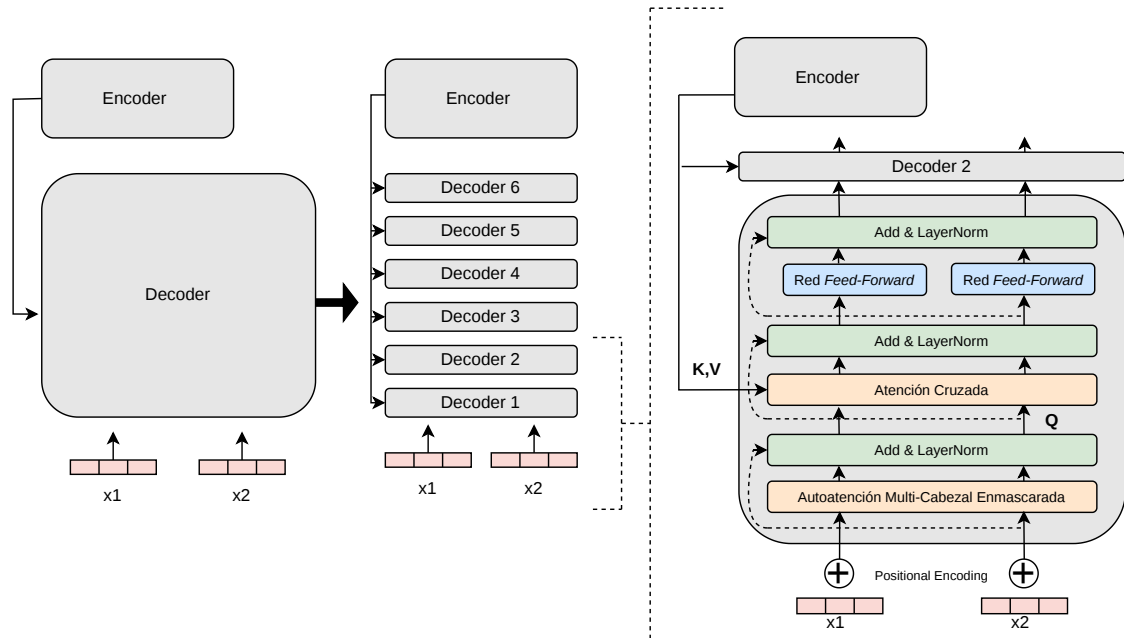


Figura 2.3: Estructura interna de un bloque del decodificador.

2.3. Atención: Mecanismo de Self-Attention

A continuación, presentamos una función clave en el funcionamiento del Transformer: la función de atención. Intuitivamente, este mecanismo permite que cada palabra de la secuencia “pregunte” a las demás cuánta información necesita de ellas para construir su representación contextualizada. Para ello, cada token genera tres vectores: una **consulta** (*query*), que representa lo que el token busca; una **clave** (*key*), que indica qué información ofrece para ser encontrado; y un **valor** (*value*), que contiene la información semántica que se transmite. La función de atención compara cada consulta con todas las claves para determinar cuánto de cada valor se incorpora a la representación final.

Definición 3. Sea $\mathbf{q} \in \mathbb{R}^{d_k}$ un vector de consulta, y sea $\{(\mathbf{k}_j, \mathbf{v}_j)\}_{j=1}^n$ un conjunto de n pares clave-valor, con $\mathbf{k}_j \in \mathbb{R}^{d_k}$ y $\mathbf{v}_j \in \mathbb{R}^{d_v}$. Una **función de atención** calcula una salida $\mathbf{z} \in \mathbb{R}^{d_v}$ como una suma ponderada

de los valores:

$$\mathbf{z} = \sum_{j=1}^n \alpha_j \mathbf{v}_j \quad (2.1)$$

donde los pesos de atención α_j se obtienen aplicando una función score : $\mathbb{R}^{d_k} \times \mathbb{R}^{d_k} \rightarrow \mathbb{R}$ que mide la compatibilidad entre la consulta y cada clave, seguida de una normalización:

$$\alpha_j = \frac{\exp(\text{score}(\mathbf{q}, \mathbf{k}_j))}{\sum_{i=1}^n \exp(\text{score}(\mathbf{q}, \mathbf{k}_i))} = \text{softmax}(\text{score}(\mathbf{q}, \mathbf{k}_j)) \quad (2.2)$$

Este mecanismo permite que el modelo decida dinámicamente cuánto debe influir cada parte de la secuencia de entrada en la representación de la palabra actual. Es decir, una alta compatibilidad entre la consulta y una clave implicará que esa palabra específica aporta información contextual crítica.

2.3.1. Atención de Producto Escalar Escalado

La entrada se compone de un vector de consultas y claves de dimensión d_k , y de uno de valores de dimensión d_v . Para calcular la atención que se debe prestar a cada elemento, primero se calcula el producto escalar de la consulta con todas las claves. Después se escala cada uno de ellos, haciendo el producto por $\frac{1}{\sqrt{d_k}}$, y posteriormente se aplica la función *softmax* (véase Apéndice B para su desarrollo completo) para obtener los pesos de los valores. Finalmente, estos pesos se utilizan para calcular una suma ponderada de los valores. A este proceso se le denomina **Atención de Producto Escalar Escalado** (*Scaled Dot-Product Attention*).

En la práctica, se introduce en la función de atención un conjunto de consultas simultáneamente, agrupadas todas juntas en una matriz Q . Las claves y los valores también se agrupan juntos en matrices K y V , por lo que la expresión para calcular la matriz de salidas es:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.3)$$

Como ya se ha mencionado, la Ecuación (2.3) incluye un factor de escala $1/\sqrt{d_k}$ en el argumento de la función *softmax*. Este factor no es arbitrario, sino que resulta necesario para evitar que la función *softmax* se sature cuando la dimensión d_k sea elevada. La siguiente proposición formaliza esta necesidad:

Proposición 2.1 (Saturación de la atención sin escalar). Sean $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ vectores cuyas componentes son variables aleatorias independientes con media 0 y varianza 1. Entonces, el producto escalar $\mathbf{q} \cdot \mathbf{k}$ tiene media 0 y varianza d_k .

Demostración. Dado que las componentes son independientes con media 0 y varianza 1:

$$\mathbb{E}[\mathbf{q} \cdot \mathbf{k}] = \sum_{j=1}^{d_k} \mathbb{E}[q_j k_j] = \sum_{j=1}^{d_k} \mathbb{E}[q_j] \mathbb{E}[k_j] = 0$$

Para la varianza, por definición:

$$\text{Var}[q_j k_j] = \mathbb{E}[q_j^2 k_j^2] - (\mathbb{E}[q_j k_j])^2 = \mathbb{E}[q_j^2] \mathbb{E}[k_j^2]$$

Dado que

$$\mathbb{E}[q_j^2] = \text{Var}[q_j] + (\mathbb{E}[q_j])^2 = 1 + 0 = 1$$

y análogamente $\mathbb{E}[k_j^2] = 1$, se tiene:

$$\text{Var}[\mathbf{q} \cdot \mathbf{k}] = \sum_{j=1}^{d_k} \text{Var}[q_j k_j] = \sum_{j=1}^{d_k} 1 = d_k$$

□

De esta forma, $\text{Var}[\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d_k}}] = (\frac{1}{\sqrt{d_k}})^2 \cdot d_k = 1$, manteniendo las puntuaciones en el rango operativo de la función *softmax*.

Ejemplo 1. Supongamos un escenario simplificado, teniendo como entrada la frase: “Escribe un email”.

En este caso, se tendrán las siguientes dimensiones:

- Dimensión de entrada (*embeddings*): $d_{model} = 4$
- Dimensión de las claves/consultas: $d_k = 4$
- Dimensión de los valores: $d_v = 3$
- Número de tokens (longitud de la secuencia): 3

Nota: En la arquitectura original del Transformer se suele utilizar $d_{model} = 512$ y se proyecta a dimensiones menores como $d_k = d_v = 64$.

Supongamos que los *embeddings* de nuestras tres palabras (“Escribe”, “un”, “email”) forman una matriz de entrada X de dimensión $3 \times d_{model}$ (con $d_{model} = 4$). Utilizaremos valores que simulan un espacio vectorial continuo:

$$X = \begin{bmatrix} 2 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix} \begin{array}{l} \leftarrow \text{“Escribe”} \\ \leftarrow \text{“un”} \\ \leftarrow \text{“email”} \end{array}$$

El modelo emplea tres matrices de pesos aprendibles (W^Q, W^K, W^V) para proyectar X en los espacios de consultas, claves y valores. Supongamos que, tras el entrenamiento con un conjunto de datos preseleccionado, el modelo ha aprendido los siguientes pesos (donde W^V reduce la dimensión a $d_v = 3$):

$$W^Q = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad W^K = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}, \quad W^V = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 20 \\ 0 & 0 & 10 \end{bmatrix}$$

Multiplicamos la matriz de entrada X por cada matriz de pesos ($Q = X \cdot W^Q$, $K = X \cdot W^K$ y $V = X \cdot W^V$). Observemos que la matriz de valores V resultante otorga valores numéricos muy distinguibles a cada token:

$$Q = \begin{bmatrix} 2 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 0 & 0 \\ 2 & 0 & 1 & -1 \end{bmatrix}, \quad V = \begin{bmatrix} 10 & 0 & 10 \\ 0 & 10 & 0 \\ 5 & 0 & 50 \end{bmatrix}$$

A continuación, calculamos el producto escalar de las consultas por las claves traspuestas para evaluar la afinidad cruzada de cada elemento de la secuencia:

$$QK^T = \begin{bmatrix} 2 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & 0 \\ 2 & 0 & 1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 3 & 0 & 6 \\ 0 & 1 & 0 \\ 6 & 0 & 3 \end{bmatrix}$$

Al analizar la primera fila (la consulta de “Escribe”), vemos que obtiene una puntuación moderada consigo misma (3), nula con “un” (0) y una afinidad muy alta (6) con “email”.

Para evitar la saturación de la función *softmax* cuando d_k es elevado, dividimos la matriz por $\sqrt{d_k} = 2$ y aplicamos dicha función fila por fila. Esto convierte la puntuación dada en una distribución de probabilidad (pesos de atención A):

$$\frac{QK^T}{2} = \begin{bmatrix} 1,5 & 0 & 3,0 \\ 0 & 0,5 & 0 \\ 3,0 & 0 & 1,5 \end{bmatrix} \xrightarrow{\text{softmax}} A = \text{softmax} \left(\frac{QK^T}{\sqrt{4}} \right) = \begin{bmatrix} 0,1753 & 0,0391 & 0,7856 \\ 0,2741 & 0,4519 & 0,2741 \\ 0,7856 & 0,0391 & 0,1753 \end{bmatrix}$$

Finalmente, calculamos la matriz de salidas Z como el producto matricial de nuestra matriz de atención A con V :

$$Z = A \cdot V = \begin{bmatrix} 0,1753 & 0,0391 & 0,7856 \\ 0,2741 & 0,4519 & 0,2741 \\ 0,7856 & 0,0391 & 0,1753 \end{bmatrix} \cdot \begin{bmatrix} 10 & 0 & 10 \\ 0 & 10 & 0 \\ 5 & 0 & 50 \end{bmatrix} = \begin{bmatrix} 5,6809 & 0,3911 & 41,0328 \\ 4,1110 & 4,5186 & 16,4441 \\ 8,7324 & 0,3911 & 16,6205 \end{bmatrix} \begin{matrix} \leftarrow \text{“Escribe”} \\ \leftarrow \text{“un”} \\ \leftarrow \text{“email”} \end{matrix}$$

Para comprender el resultado obtenido, analicemos la evolución del vector correspondiente a la palabra “Escribe” (primera fila de las matrices) desde su estado inicial hasta su representación final:

- **Representación aislada (Matriz V):** Originalmente, el valor del token “Escribe” venía dado por el vector $[10, 0, 10]$, mientras que el token “email” correspondía a $[5, 0, 50]$. Si asumimos que la tercera dimensión de este espacio vectorial codifica información semántica relacionada con el ámbito digital, observamos que el sustantivo “email” presenta un valor mucho mayor (50) frente a la naturaleza genérica del verbo “Escribe” (10).
- **Distribución de la atención (Matriz A):** La primera fila de la matriz de atención normalizada indica cómo se distribuye el flujo de información. Para redefinir la palabra “Escribe”, el modelo le otorga un peso del 0,1753 (17,53 %) a sí misma, un 0,0391 (3,91 %) a la palabra “un” y un destacable 0,7856 (78,56 %) a la palabra “email”.
- **Representación contextualizada (Matriz Z):** En la matriz de salida $Z = A \cdot V$, el nuevo vector para la palabra “Escribe” pasa a ser $\mathbf{z} = [5,6809 \quad 0,3911 \quad 41,0328]$.

El vector resultante de la matriz Z ya no representa el verbo genérico “escribir”, sino que ha cambiado para incorporar el contexto de su secuencia. Al absorber gran parte del valor en la tercera dimensión proveniente de la palabra “email” (alcanzando un 41,0328), la nueva representación matemática transmite a las capas posteriores de la red neuronal que la acción está íntimamente ligada a un medio digital.

De esta manera, el modelo Transformer tiene capacidad para resolver dependencias contextuales complejas y ambigüedades, siendo capaz de generar representaciones vectoriales distintas para el mismo verbo en frases como “Escribe un email” frente a “Escribe un poema”.

2.3.2. Atención Multi-Cabezal

En lugar de realizar una única función de atención utilizando consultas, claves y valores con la dimensión original del modelo (d_{model}), Vaswani et al. [5] propusieron proyectar linealmente Q , K y V múltiples veces en subespacios de menor dimensión. Específicamente, se realizan h proyecciones independientes, cada una con sus propias matrices de pesos aprendidas.

Sobre cada una de estas versiones proyectadas se calcula la atención de producto escalar escalado en paralelo, generando h salidas independientes. Finalmente, estas salidas se concatenan y se proyectan linealmente para obtener el resultado final. A este proceso se le denomina **Atención Multi-Cabezal** (*Multi-Head Attention*), y se define como:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \quad (2.4)$$

donde cada cabezal (*head*) se calcula de forma independiente como:

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V) \quad (2.5)$$

con matrices de proyección $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ propias de cada cabezal i , y donde $X \in \mathbb{R}^{n \times d_{model}}$ representa la entrada a la sub-capa (en el caso de la autoatención, X corresponde a la secuencia de representaciones de la capa anterior). Finalmente, las salidas se combinan mediante una matriz compartida $W^O \in \mathbb{R}^{hd_v \times d_{model}}$.

La dimensión de cada cabezal se elige como $d_k = d_v = d_{model}/h$, de modo que la concatenación de los h cabezales recupera la dimensión original: $h \cdot d_v = h \cdot \frac{d_{model}}{h} = d_{model}$. En consecuencia, $W^O \in \mathbb{R}^{d_{model} \times d_{model}}$ y el coste computacional total es comparable al de un único cabezal con la dimensión completa.

Limitaciones de un único cabezal

Un único cabezal de atención calcula una matriz de pesos $A \in \mathbb{R}^{n \times n}$ (Ecuación 2.3) que se aplica por igual a todas las dimensiones del espacio de valores. Es decir, la representación contextualizada $\mathbf{z}_i \in \mathbb{R}^{d_v}$ de cada token i se construye ponderando los vectores con un único conjunto de pesos α_{ij} :

$$\mathbf{z}_i = \sum_{j=1}^n \alpha_{ij} \mathbf{v}_j$$

Cada peso α_{ij} determina cuánta información del token j se incorpora a la representación del token i , y este mismo peso se aplica de forma idéntica a todas las componentes de $\mathbf{v}_j$. Con h cabezales, el modelo dispone de h matrices de pesos independientes $A_1, \dots, A_h$, cada una operando en un subespacio distinto de $\mathbb{R}^{d_k}$. De este modo, cada cabezal puede especializarse en capturar un tipo diferente de relación sin interferir con los demás, y la proyección final W^O combina toda esta información. El siguiente ejemplo ilustra este efecto.

Ejemplo 2. Retomemos la frase “Escribe un email” del Ejemplo 1, ahora con $h = 2$ cabezales de dimensión $d_k = d_v = 2$ cada uno. Cada cabezal proyecta la entrada X a un subespacio diferente de $\mathbb{R}^2$ mediante sus propias matrices de pesos. Supongamos que, tras el entrenamiento, el modelo ha aprendido las siguientes proyecciones:

$$\begin{aligned} \text{Cabezal 1: } W_1^Q &= \begin{bmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}, & W_1^K &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, & W_1^V &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix} \\ \text{Cabezal 2: } W_2^Q &= \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}, & W_2^K &= \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, & W_2^V &= \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Observemos que $W_1^Q \neq W_1^K$ y $W_2^Q \neq W_2^K$: cada cabezal “pregunta” con unas dimensiones del *embedding* y “busca” con otras. En concreto, el cabezal 1 construye las consultas a partir de las dimensiones 1 y 3 del *embedding*, pero compara con las claves extraídas de las dimensiones 3 y 4. El cabezal 2 construye las consultas con las dimensiones 2 y 4, y las claves con las dimensiones 1 y 2.

Multiplicando la entrada X (empleamos la misma que la del ejemplo anterior, $d_{model} = 4$) por cada par de matrices, obtenemos las consultas y claves de cada cabezal:

$$\begin{aligned} X &= \begin{bmatrix} 2 & 0 & 1 & -1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 2 & 1 \end{bmatrix} \begin{array}{l} \leftarrow \text{“Escribe”} \\ \leftarrow \text{“un”} \\ \leftarrow \text{“email”} \end{array} \\ Q_1 &= \begin{bmatrix} 2 & 1 \\ 0 & 0 \\ 1 & 2 \end{bmatrix}, & K_1 &= \begin{bmatrix} 1 & -1 \\ 0 & 0 \\ 2 & 1 \end{bmatrix}, & Q_2 &= \begin{bmatrix} 0 & -1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}, & K_2 &= \begin{bmatrix} 2 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \end{aligned}$$

Calculamos las puntuaciones de afinidad y aplicamos la función *softmax* (con $\sqrt{d_k} = \sqrt{2}$) para obtener los pesos de atención de cada cabezal:

$$\begin{aligned} A_1 &= \text{softmax}\left(\frac{Q_1 K_1^T}{\sqrt{2}}\right) = \begin{bmatrix} 0,0543 & 0,0268 & 0,9189 \\ 0,3333 & 0,3333 & 0,3333 \\ 0,0268 & 0,0543 & 0,9189 \end{bmatrix} \\ A_2 &= \text{softmax}\left(\frac{Q_2 K_2^T}{\sqrt{2}}\right) = \begin{bmatrix} 0,4011 & 0,1978 & 0,4011 \\ 0,5760 & 0,1400 & 0,2840 \\ 0,2483 & 0,5035 & 0,2483 \end{bmatrix} \end{aligned}$$

Analicemos ahora el contraste entre ambas matrices. Si nos centramos en la palabra “Escribe” (primera fila):

- En el **cabezal 1**, concentra el 91,9 % de su atención en “email” y apenas un 5,4 % en sí misma. Este cabezal ha capturado la relación semántica entre la acción y su objeto directo.
- En el **cabezal 2**, distribuye la atención de forma equilibrada: un 40,1 % a sí misma, un 40,1 % a “email” y un 19,8 % a “un”. Este cabezal no se concentra en una única relación, sino que recoge información del contexto en general.

El comportamiento de “email” (tercera fila) también resulta interesante: en el cabezal 1 se atiende a sí misma con un 91,9 %, mientras que en el cabezal 2 dirige un 50,3 % de su atención hacia el artículo “un” que la precede.

Para completar el cálculo, obtenemos la salida de cada cabezal multiplicando los pesos de atención por sus respectivos valores ($V_1 = XW_1^V$, $V_2 = XW_2^V$):

$$\text{head}_1 = A_1 \cdot V_1 = \begin{bmatrix} 1,0275 & 0,0268 \\ 1,0000 & 0,3333 \\ 0,9725 & 0,0543 \end{bmatrix}, \quad \text{head}_2 = A_2 \cdot V_2 = \begin{bmatrix} 1,2033 & 0,0000 \\ 1,1440 & -0,2920 \\ 0,7448 & 0,0000 \end{bmatrix}$$

Finalmente, concatenamos ambas salidas para recuperar la dimensión original $d_{model} = 4$ y proyectamos con W^O :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2) W^O = \begin{bmatrix} 1,0275 & 0,0268 & 1,2033 & 0,0000 \\ 1,0000 & 0,3333 & 1,1440 & -0,2920 \\ 0,9725 & 0,0543 & 0,7448 & 0,0000 \end{bmatrix} W^O$$

Las dos primeras columnas provienen del cabezal 1 (que ha capturado la relación acción-objeto) y las dos últimas del cabezal 2 (que ha recogido el contexto local). La matriz $W^O \in \mathbb{R}^{4 \times 4}$ (ya que $hd_v = 2 \cdot 2 = 4$ y $d_{model} = 4$), aprendida durante el entrenamiento, combinará ambas perspectivas en una representación final unificada que conserva información de ambos tipos de relación.

Notemos que, gracias a la proyección final W^O , la salida $\text{MultiHead}(Q, K, V) \in \mathbb{R}^{n \times d_{model}}$ recupera la dimensión original de la entrada X independientemente de las dimensiones internas d_k y d_v elegidas para cada cabezal. Esto garantiza la compatibilidad con la conexión residual que envuelve a cada sub-cap.

2.3.3. Aplicaciones de la Atención en el Modelo

El Transformer utiliza atención multi-cabezal de tres formas diferentes, que se distinguen por la procedencia de las matrices Q , K y V :

- **Atención Cruzada Codificador-Decodificador (Cross-Attention):** Las consultas Q son el resultado de proyectar la salida de la capa anterior del decodificador, mientras que las claves K y valores V se obtienen al proyectar la salida final del codificador. Esto permite que cada posición del decodificador atienda a todas las posiciones de la secuencia de entrada, conectando la representación codificada con el proceso de generación.
- **Autoatención en el Codificador:** Las consultas, claves y valores provienen del mismo lugar: la salida de la capa anterior del codificador. Cada posición puede así atender a todas las demás posiciones de la secuencia, captando el contexto global de la entrada.
- **Autoatención enmascarada en el Decodificador:** Idéntica a la anterior, pero con una restricción fundamental derivada de la naturaleza auto-regresiva del modelo: la predicción en la posición i solo puede depender de las posiciones anteriores. Para imponer esta restricción, se aplica una máscara antes de la función *softmax*, redefiniendo la función de atención como:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right) V \quad (2.6)$$

donde $M \in \mathbb{R}^{n \times n}$ es la matriz de enmascaramiento. Los índices i y j representan, respectivamente, la posición del token que realiza la consulta (fila) y la posición del token al que se atiende (columna). La matriz se define como:

$$M_{ij} = \begin{cases} 0 & \text{si } j \leq i \\ -\infty & \text{si } j > i \end{cases} \quad (2.7)$$

Al sumar M a las puntuaciones, las posiciones futuras ($j > i$) reciben un valor de $-\infty$ que, tras la función *softmax*, se convierte en un peso de atención nulo. De este modo, se garantiza que el token en la posición i solo atiende a los tokens en las posiciones $\{1, \dots, i\}$.

2.4. Codificación Posicional

El mecanismo de autoatención descrito en la Sección 2.3 es invariante a permutaciones. Dado que ninguna de las operaciones que componen la función de atención (Ecuación 2.3) incorpora información sobre la posición de los tokens, su salida es equivariante al orden de la secuencia, lo que significa que la representación generada para cada token es independiente de la posición que ocupe. Dicho de otro modo, las frases “el gato come pescado” y “pescado come gato el” producen exactamente la misma representación, solo que permutadas.

Esta observación se puede formalizar, pues recordemos que una permutación de los n tokens de la secuencia se puede representar mediante una matriz de permutación $P \in \mathbb{R}^{n \times n}$, que satisface $P^T P = I$ (es decir, P es ortogonal con $P^{-1} = P^T$). Multiplicar la matriz de entrada X por P a la izquierda reordena sus filas, y por tanto el orden de los tokens.

Proposición 2.2 (Equivariancia de la autoatención a permutaciones). Sea $X \in \mathbb{R}^{n \times d_{\text{model}}}$ la matriz de entrada y $P \in \mathbb{R}^{n \times n}$ una matriz de permutación cualquiera. Entonces:

$$\text{Attention}(PX) = P \cdot \text{Attention}(X) \quad (2.8)$$

Es decir, permutar los tokens de la entrada produce la misma permutación en la salida, sin alterar las representaciones contextualizadas.

Demostración. Denotemos por $Q = XW^Q$, $K = XW^K$ y $V = XW^V$ las proyecciones de la entrada original. Si permutamos la entrada, las nuevas proyecciones son:

$$Q' = PXW^Q = PQ, \quad K' = PXW^K = PK, \quad V' = PXW^V = PV$$

Calculamos la matriz de puntuaciones de la entrada permutada:

$$\frac{Q'K'^T}{\sqrt{d_k}} = \frac{(PQ)(PK)^T}{\sqrt{d_k}} = \frac{PQK^T P^T}{\sqrt{d_k}} = P \left(\frac{QK^T}{\sqrt{d_k}} \right) P^T$$

Dado que la función *softmax* se aplica fila por fila, y multiplicar por P a la izquierda permuta las filas mientras que multiplicar por P^T a la derecha permuta las columnas, se cumple que:

$$\text{softmax}(P \cdot A \cdot P^T) = P \cdot \text{softmax}(A) \cdot P^T$$

donde $A = QK^T / \sqrt{d_k}$. Denotando $S = \text{softmax}(A)$, la salida de la atención permutada es:

$$\text{Attention}(PX) = (P \cdot S \cdot P^T) (PV) = P \cdot S \cdot \underbrace{P^T P}_I \cdot V = P \cdot S \cdot V = P \cdot \text{Attention}(X) \quad \square$$

Este resultado demuestra que, sin codificación posicional, la autoatención trata la entrada como un conjunto no ordenado y no como una secuencia: las representaciones contextualizadas de cada token son idénticas independientemente del orden en que se presenten.

Para que el modelo pueda distinguir el orden de la secuencia, se suma a cada *embedding* de entrada un vector de **codificación posicional PE** que, como su nombre indica, codifica la posición del token en la secuencia. Así, la entrada real a la primera capa del Transformer es:

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{PE}_{pos} \in \mathbb{R}^{d_{model}}$$

donde $\mathbf{e}_i \in \mathbb{R}^{d_{model}}$ es el *embedding* del token y $\mathbf{PE}_{pos} \in \mathbb{R}^{d_{model}}$ su codificación posicional.

Definición 4. Sea $pos \in \mathbb{N}$ la posición del token en la secuencia e $i \in \{0, 1, \dots, d_{model}/2 - 1\}$ el índice de la dimensión. La **codificación posicional seno/coseno** se define como:

$$\begin{aligned} PE_{(pos, 2i)} &= \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \\ PE_{(pos, 2i+1)} &= \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \end{aligned} \quad (2.9)$$

Cada par de dimensiones consecutivas $(2i, 2i+1)$ corresponde a un seno/coseno de frecuencia $\omega_i = 1/10000^{2i/d_{model}}$. A medida que el índice i crece, la frecuencia decrece: las primeras dimensiones oscilan rápidamente (capturando posiciones cercanas), mientras que las últimas oscilan lentamente (capturando posiciones a larga distancia).

Propiedad de los desplazamientos relativos

La elección de este tipo de funciones no es arbitraria. Su ventaja fundamental es que la codificación de una posición desplazada $pos + k$ puede expresarse como una transformación lineal de la codificación en pos , donde la transformación depende únicamente del desplazamiento k .

Proposición 2.3 (Linealidad del desplazamiento posicional). Para cualquier desplazamiento fijo $k \in \mathbb{Z}$ y cualquier índice de dimensión i , se cumple:

$$\begin{pmatrix} PE_{(pos+k, 2i)} \\ PE_{(pos+k, 2i+1)} \end{pmatrix} = \begin{pmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{pmatrix} \begin{pmatrix} PE_{(pos, 2i)} \\ PE_{(pos, 2i+1)} \end{pmatrix} \quad (2.10)$$

donde $\omega_i = 1/10000^{2i/d_{model}}$. Es decir, el desplazamiento posicional corresponde a una rotación en el plano $(2i, 2i+1)$ cuyo ángulo depende solo de k , no de pos .

Demostración. Aplicamos las fórmulas de suma de ángulos del seno y el coseno con $\alpha = \omega_i \cdot pos$ y $\beta = \omega_i \cdot k$, y posteriormente la Definición 4:

$$\begin{aligned} PE_{(pos+k, 2i)} &= \sin(\omega_i(pos+k)) \\ &= \sin(\omega_i pos) \cos(\omega_i k) + \cos(\omega_i pos) \sin(\omega_i k) \\ PE_{(pos+k, 2i+1)} &= \cos(\omega_i(pos+k)) \\ &= \cos(\omega_i pos) \cos(\omega_i k) - \sin(\omega_i pos) \sin(\omega_i k) \end{aligned} \quad \square$$

Este resultado implica que el modelo puede aprender relaciones entre posiciones relativas mediante transformaciones lineales de las codificaciones, sin necesidad de memorizar cada posición absoluta individualmente.

En la Figura 2.4 se puede apreciar la oscilación de las distintas dimensiones de la codificación posicional: las primeras dimensiones (zona izquierda) oscilan con alta frecuencia, lo que permite al modelo

distinguir tokens cercanos. Por el contrario, las últimas dimensiones (zona derecha) oscilan con frecuencias extremadamente bajas, pero diferenciando claramente tokens separados por largas distancias, lo cual se puede apreciar principalmente en el mapa de calor de los 1000 tokens.

Así, cada posición queda codificada por una combinación única de funciones seno y coseno, proporcionando al Transformer una representación posicional variada sin introducir parámetros adicionales al modelo.

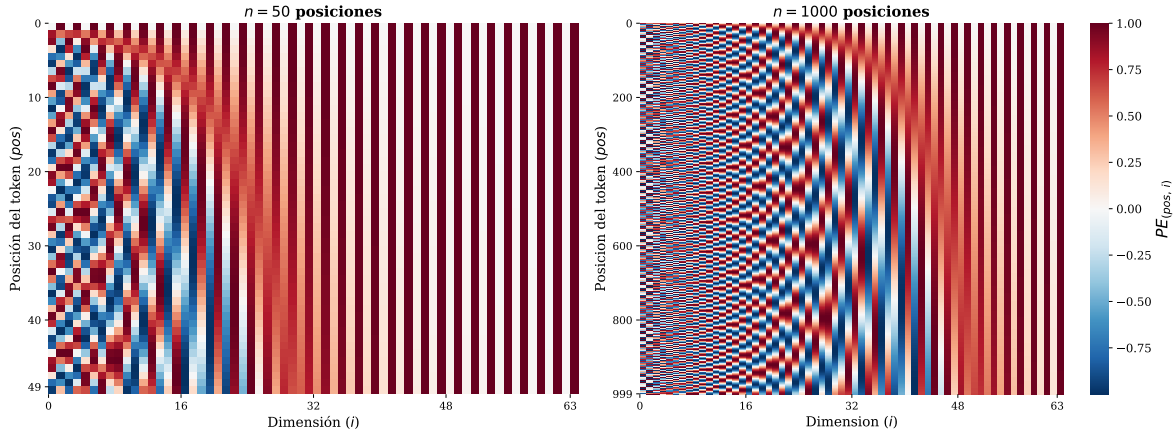


Figura 2.4: Mapas de calor de la codificación posicional seno/coseno para $n = 50$ y $n = 1000$ tokens respectivamente, ambas con $d_{model} = 64$ dimensiones.

2.5. Estabilidad del Gradiente en el Transformer

Como se ha visto en la Sección 2.1, el desvanecimiento del gradiente ha sido la limitación central de las arquitecturas previas. El Transformer lo aborda mediante dos mecanismos complementarios: las conexiones residuales, que proporcionan un camino directo para el flujo del gradiente, y la normalización de capa, que estabiliza la escala de las activaciones. Analizamos cada uno por separado.

2.5.1. Conexiones Residuales

En una red neuronal convencional, como las descritas en el Capítulo 1, cada capa l transforma su entrada mediante una función aprendida $f^{(l)}$, de modo que la salida de dicha capa es $\mathbf{a}^{(l)} = f^{(l)}(\mathbf{a}^{(l-1)})$ (Ecuación 1.3). Como se demostró en la Ecuación (1.5), al retropropagar el gradiente a través de una cadena de estas transformaciones, la regla de la cadena produce un producto de matrices jacobianas cuya norma puede atenuarse exponencialmente.

He et al. [6] propusieron una modificación arquitectónica que altera de forma fundamental este flujo. En lugar de aprender directamente la transformación deseada, cada capa aprende únicamente la diferencia, el **residuo**, respecto a su entrada. En el contexto del Transformer, denotaremos por $\mathcal{F}^{(l)}$ la transformación aprendida en cada sub-capa (ya sea de atención o *feed-forward*) y por $\mathbf{x}^{(l)}$ su salida, para distinguirla de las capas densas convencionales del Capítulo 1. Formalmente:

Definición 5. Sea $\mathcal{F}^{(l)}$ la transformación aprendida en la sub-capa l del Transformer (ya sea la sub-capa de atención o la red *feed-forward*). Una **conexión residual** define la salida de dicha sub-capa como la suma de la entrada original con la transformación aplicada sobre ella:

$$\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)}) \quad (2.11)$$

Es decir, la red ya no necesita aprender la función completa que transforma la entrada en la salida, sino únicamente el **residuo** $\mathcal{F}^{(l)}(\mathbf{x}^{(l-1)}) = \mathbf{x}^{(l)} - \mathbf{x}^{(l-1)}$. Si la transformación óptima fuera la identidad (es decir, no modificar la entrada), a la red le bastaría con aprender $\mathcal{F}^{(l)} \approx \mathbf{0}$, lo cual es significativamente más

sencillo y tiene un menor coste computacional que aprender una función identidad completa mediante multiplicaciones matriciales.

Sin embargo, la verdadera ventaja de este mecanismo se revela al analizar el flujo del gradiente durante la retropropagación. La siguiente proposición formaliza este resultado:

Proposición 2.4 (Flujo del gradiente con conexión residual). Sea $\mathcal{L}$ la función de pérdida de la red y sea la salida de la capa l definida según la Ecuación (2.11). Entonces, el gradiente de la pérdida respecto a la entrada de dicha capa satisface:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right) \quad (2.12)$$

donde I denota la matriz identidad.

Demostración. Partimos de la Ecuación (2.11) y calculamos la derivada parcial de la salida $\mathbf{x}^{(l)}$ respecto a la entrada $\mathbf{x}^{(l-1)}$. Dado que $\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})$, por la linealidad de la derivada obtenemos:

$$\frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{x}^{(l-1)}} = I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}}$$

Ahora aplicamos la regla de la cadena para obtener el gradiente de la función de pérdida. Como $\mathcal{L}$ depende de $\mathbf{x}^{(l-1)}$ a través de $\mathbf{x}^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \cdot \frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right)$$

como se quería demostrar. $\square$

Notemos que la presencia del término identidad I en la Ecuación (2.12) es la clave del resultado. Desarrollando el producto:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l-1)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \cdot I}_{\text{camino directo}} + \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \cdot \frac{\partial \mathcal{F}^{(l)}}{\partial \mathbf{x}^{(l-1)}}}_{\text{camino a través de } \mathcal{F}^{(l)}}$$

El primer sumando es simplemente $\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}}$ sin modificar: un camino directo de la capa l a la anterior. Incluso si la derivada de $\mathcal{F}^{(l)}$ es próxima a cero, el término I garantiza que el gradiente se retropropague íntegramente sin atenuarse. Este resultado se generaliza a una red completa de L capas residuales:

Corolario 2.1 (Flujo de gradiente en L capas residuales). En una red de L capas con conexiones residuales, el gradiente de la pérdida respecto a la entrada de la red satisface:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \prod_{l=1}^L \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right) \quad (2.13)$$

Demostración. Se obtiene aplicando la Proposición 2.4 de forma iterativa desde la capa L hasta la primera capa $l = 1$:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \cdot \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{x}^{(L-1)}} \cdot \frac{\partial \mathbf{x}^{(L-1)}}{\partial \mathbf{x}^{(L-2)}} \cdots \frac{\partial \mathbf{x}^{(1)}}{\partial \mathbf{x}^{(0)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \prod_{l=1}^L \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right)$$

$\square$

Para comprender la importancia de este resultado, conviene compararlo directamente con la situación analizada en el Capítulo 1. En una red profunda sin conexiones residuales, el gradiente respecto a las primeras capas involucra el producto:

$$\prod_{m=2}^M \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}}$$

donde demostramos que, bajo la hipótesis $\left\| \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right\| \leq \omega < 1$, la norma de este producto se atenúa exponencialmente según $\omega^{M-1} \rightarrow 0$ (Ecuación 1.10).

En cambio, al expandir el producto de la Ecuación (2.13), observamos que siempre aparece el término correspondiente al producto de todas las matrices identidad:

$$\prod_{l=1}^L \left(I + \frac{\partial \mathcal{F}^{(l)}}{\partial \mathbf{x}^{(l-1)}} \right) = I + \sum_{l=1}^L \frac{\partial \mathcal{F}^{(l)}}{\partial \mathbf{x}^{(l-1)}} + \dots$$

donde los términos omitidos corresponden a productos cruzados de dos o más jacobianas. El primer sumando, I , constituye un camino directo desde la salida hasta la entrada que no depende de ninguna derivada intermedia y que, por tanto, no puede atenuarse. En cambio, en una red convencional, el gradiente debe atravesar todas las capas intermedias y cada una puede atenuarlo. Con conexiones residuales, el gradiente dispone de un “atajo” que elude las transformaciones intermedias, garantizando que la señal de error llegue intacta a las primeras capas.

2.5.2. Normalización de Capa

Recordemos que la conexión residual suma la transformación a la entrada en cada capa: $\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})$. Si $\mathcal{F}^{(l)}$ produce valores positivos de forma consistente, estas sumas se acumulan capa tras capa, provocando que la magnitud de las activaciones crezca progresivamente.

Este crecimiento tiene una consecuencia directa: cada capa recibe datos con una media y varianza diferentes respecto a las de la capa anterior, lo que obliga al modelo a readaptarse continuamente a una entrada cuya escala cambia, dificultando la convergencia del entrenamiento.

Ejemplo 3. Consideremos la función sigmoide $\sigma(z) = \frac{1}{1+e^{-z}}$, cuya derivada es $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Cuando la entrada se encuentra en un rango moderado, la derivada tiene valores útiles para el aprendizaje: $\sigma'(0) = 0,25$. Sin embargo, si las activaciones crecen sin control por la acumulación de sumas residuales, la función se satura y la derivada colapsa: $\sigma'(10) \approx 0,00005$. De esta forma, el gradiente es prácticamente nulo y el aprendizaje se detiene.

Para abordar este problema, Ba et al. [7] propusieron la normalización de capa (*Layer Normalization*), una técnica que reescala las activaciones en cada sub-capas, forzando que se mantengan en un rango estable independientemente de cuántas sumas residuales se hayan acumulado.

Definición 6. Sea $\mathbf{v} = (v_1, \dots, v_d) \in \mathbb{R}^d$ un vector de entrada a la capa de normalización. La **normalización de capa** se define como la transformación $\text{LN} : \mathbb{R}^d \rightarrow \mathbb{R}^d$ dada por:

$$\text{LN}(\mathbf{v}) = \gamma \odot \frac{\mathbf{v} - \mu}{s} + \beta \quad (2.14)$$

donde $\odot$ denota el producto de Hadamard, y:

- $\mu = \frac{1}{d} \sum_{k=1}^d v_k$ es la media de las componentes del vector.
- $s = \sqrt{\frac{1}{d} \sum_{k=1}^d (v_k - \mu)^2 + \varepsilon}$ es la desviación típica, con $\varepsilon > 0$ una constante de estabilidad numérica.

- $\gamma, \beta \in \mathbb{R}^d$ son vectores de parámetros aprendibles (escala y sesgo, respectivamente).

Notemos que la resta de la media y la división por la desviación típica centran y normalizan las activaciones, forzando que cada vector tenga media cero y varianza unitaria antes del reescalado (de ahí la condición de la Proposición 2.1). Además, los parámetros aprendibles γ y β permiten que la red recupere capacidad expresiva: si la representación óptima requiere una distribución diferente a la normalizada, el modelo puede aprenderlo ajustando estos parámetros durante el entrenamiento.

Integración en el Transformer

En la arquitectura original del Transformer [5], la presentada en la Figura 2.1, la normalización de capa se aplica después de la conexión residual. Combinando la Ecuación (2.11) con la Ecuación (2.14), la salida completa de cada sub-capita del codificador se expresa como:

Definición 7. La disposición **Post-LN** (*Post-Layer Normalization*) aplica la normalización de capa tras la suma residual:

$$\mathbf{x}^{(l)} = \text{LN}\left(\mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})\right) \quad (2.15)$$

Intuitivamente, cada sub-capita primero calcula su transformación, la suma a la entrada original, y solo entonces normaliza el resultado antes de pasarlo a la siguiente capa.

Sin embargo, trabajos posteriores [8] identificaron que esta disposición presenta dificultades durante el entrenamiento de redes profundas: los gradientes de las capas cercanas a la salida pueden alcanzar magnitudes muy elevadas en la inicialización, lo que exige una fase de calentamiento (*warm-up*) del *learning rate* η para estabilizar la optimización. Como alternativa, se propuso una segunda disposición:

Definición 8. La disposición **Pre-LN** (*Pre-Layer Normalization*) aplica la normalización de capa dentro del bloque residual, antes de la transformación:

$$\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}\left(\text{LN}(\mathbf{x}^{(l-1)})\right) \quad (2.16)$$

La diferencia, aunque aparentemente menor, tiene consecuencias profundas para el flujo del gradiente. En la variante Post-LN, la normalización envuelve a toda la suma residual, lo que distorsiona el camino directo del gradiente que identificamos en la Proposición 2.4. En la variante Pre-LN, la normalización actúa únicamente sobre la entrada de $\mathcal{F}^{(l)}$, preservando la estructura aditiva $\mathbf{x}^{(l-1)} + (\cdot)$ de la conexión residual. De este modo, el término identidad I de la Ecuación (2.12) (el “atajo”) permanece sin alteración alguna. Una representación visual de dicha diferencia se encuentra en la Figura 2.5.

Es precisamente esta variante Pre-LN la que permite demostrar formalmente que los gradientes se mantienen acotados independientemente de la profundidad de la red. El resultado formal se presenta en la siguiente sección.

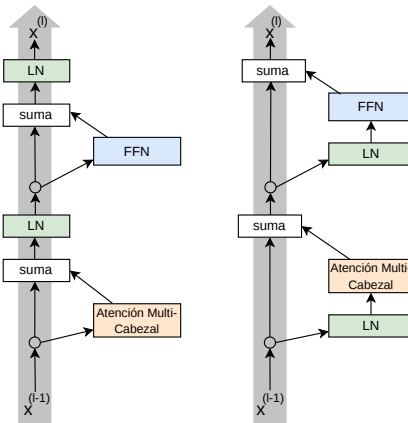


Figura 2.5: Comparativa de las disposiciones Post-LN (izquierda) y Pre-LN (derecha).

Para comprender por qué la disposición de la normalización de capa es determinante, calculemos la jacobiana de cada variante y comparémosla con el resultado de la Proposición 2.4.

Proposición 2.5 (Gradiente en Post-LN). En la variante Post-LN (Ecuación 2.15), la jacobiana de la salida respecto a la entrada es:

$$\frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \text{LN}}{\partial \mathbf{u}} \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right) \quad (2.17)$$

donde $\mathbf{u} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})$ es el argumento de la normalización.

Demostración. En Post-LN, la salida se define como $\mathbf{x}^{(l)} = \text{LN}(\mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)}))$. Sea $\mathbf{u} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})$. Aplicando la regla de la cadena:

$$\frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \text{LN}(\mathbf{u})}{\partial \mathbf{u}} \cdot \frac{\partial \mathbf{u}}{\partial \mathbf{x}^{(l-1)}} = \frac{\partial \text{LN}}{\partial \mathbf{u}} \left(I + \frac{\partial \mathcal{F}^{(l)}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \right)$$

donde en la segunda igualdad se ha aplicado el mismo razonamiento que en la demostración de la Proposición 2.4 para obtener la derivada de $\mathbf{u}$ respecto a $\mathbf{x}^{(l-1)}$. $\square$

Proposición 2.6 (Gradiente en Pre-LN). En la variante Pre-LN (Ecuación 2.16), la jacobiana de la salida respecto a la entrada es:

$$\frac{\partial \mathbf{x}^{(l)}}{\partial \mathbf{x}^{(l-1)}} = I + \frac{\partial \mathcal{F}^{(l)}}{\partial \text{LN}(\mathbf{x}^{(l-1)})} \cdot \frac{\partial \text{LN}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}} \quad (2.18)$$

Demostración. En Pre-LN, la salida se define como $\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}(\text{LN}(\mathbf{x}^{(l-1)}))$. La derivada del primer sumando respecto a $\mathbf{x}^{(l-1)}$ es I . Para el segundo sumando, aplicamos la regla de la cadena:

$$\frac{\partial}{\partial \mathbf{x}^{(l-1)}} \mathcal{F}^{(l)}(\text{LN}(\mathbf{x}^{(l-1)})) = \frac{\partial \mathcal{F}^{(l)}}{\partial \text{LN}(\mathbf{x}^{(l-1)})} \cdot \frac{\partial \text{LN}(\mathbf{x}^{(l-1)})}{\partial \mathbf{x}^{(l-1)}}$$

Sumando ambos términos se obtiene el resultado. $\square$

La comparación entre las Ecuaciones (2.17) y (2.18) revela una diferencia estructural fundamental. En **Post-LN**, la jacobiana de la normalización $\partial \text{LN} / \partial \mathbf{u}$ multiplica al producto completo, incluido el término identidad I . De este modo, el camino directo del gradiente identificado en la Proposición 2.4 queda distorsionado por la normalización: ya no existe un sumando I “puro” que garantice el flujo íntegro de la señal.

En **Pre-LN**, en cambio, el término I aparece fuera de cualquier producto con la jacobiana de la normalización. La normalización solo afecta al segundo sumando (el que pasa a través de $\mathcal{F}^{(l)}$), preservando intacto el camino directo. Esto es exactamente la misma estructura de la Ecuación (2.12), con la garantía adicional de que la normalización estabiliza la escala de las activaciones dentro de $\mathcal{F}^{(l)}$.

2.5.3. Gradientes Bien Comportados en el Transformer

Hasta el momento se ha demostrado que las conexiones residuales proporcionan un camino directo para el gradiente (Proposición 2.4), que la normalización de capa estabiliza la escala de las activaciones (Ecuación 2.14), y que la disposición Pre-LN preserva dicho camino directo, a diferencia de la Post-LN (Proposiciones 2.5 y 2.6). Queda por demostrar que estos mecanismos, al actuar conjuntamente, garantizan formalmente que el desvanecimiento del gradiente no se produce.

Xiong et al. [8] dieron respuesta afirmativa a esta cuestión. Dado que los pesos del Transformer se inicializan de forma aleatoria (condición 2 del Teorema 2.1), el gradiente resultante es también una variable aleatoria.

Teorema 2.1 (Gradientes acotados en el Pre-LN Transformer, Xiong et al. (2020)). *Consideremos un Transformer con Pre-Layer Normalization compuesto por L bloques. Cada bloque l contiene sub-capas de autoatención (Self-Attention) y redes feed-forward (FFN), todas ellas con la estructura Pre-LN definida en la Ecuación (2.16):*

$$\mathbf{x}^{(l)} = \mathbf{x}^{(l-1)} + \mathcal{F}^{(l)}\left(\text{LN}(\mathbf{x}^{(l-1)})\right)$$

Supongamos las siguientes condiciones de inicialización:

1. *Los parámetros de escala y sesgo de la normalización de capa se inicializan como $\gamma = \mathbf{1}$ y $\beta = \mathbf{0}$, respectivamente.*
2. *Todas las matrices de pesos de las sub-capas de atención (W_i^Q, W_i^K, W_i^V, W^O) y de las redes FFN (W_1, W_2) se inicializan siguiendo una distribución simétrica con media 0 y varianza $1/d$, donde d es la dimensión del modelo.*
3. *Los vectores de sesgo se inicializan a cero.*

Entonces $\exists C > 0$ tal que, en la inicialización, la norma esperada del gradiente de la función de pérdida $\mathcal{L}$ respecto a la entrada de cada bloque l satisface:

$$\mathbb{E} \left[\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \right\| \right] \leq C \quad \forall l = 1, \dots, L \quad (2.19)$$

Es decir, la norma del gradiente está acotada por una constante C en todos los bloques de la red, sin depender de la profundidad total L .

Demostración. En la demostración se estudia la propagación de la varianza del gradiente capa a capa, demostrando que la estructura Pre-LN mantiene dicha varianza acotada independientemente de la profundidad. Esta puede consultarse en Xiong et al. [8], Theorem 1 y Lemmas 1–3. $\square$

Este teorema cierra el argumento que ha servido de hilo conductor durante todo el documento. Recordemos que en el Capítulo 1 demostramos que para una red profunda sin conexiones residuales ni normalización, la norma del gradiente en las primeras capas se atenúa exponencialmente con la profundidad (Ecuación 1.10):

$$\left\| \prod_{m=2}^M \frac{\partial \mathbf{a}^{(m)}}{\partial \mathbf{a}^{(m-1)}} \right\| \leq \omega^{M-1} \xrightarrow{M \rightarrow \infty} 0$$

En el Transformer con Pre-LN, el Teorema 2.1 garantiza la situación opuesta:

$$\mathbb{E} \left[\left\| \frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(l)}} \right\| \right] \leq C \quad \text{independientemente de } L$$

La diferencia es estructural: en el primer caso, el gradiente decae exponencialmente conforme aumenta la profundidad; en el segundo, permanece acotado por una constante. Esto significa que todas las capas del Transformer reciben una señal de error de magnitud comparable, permitiendo que el aprendizaje se distribuya de forma efectiva a lo largo de toda la red.

A modo de resumen, la Tabla 2.1 agrupa las propiedades de las cuatro arquitecturas estudiadas a lo largo del documento.

Arquitectura	Camino entre tokens	Gradiente	Paralelizable
Feed-Forward	—	$\ \nabla\ \leq \omega^{M-1} \rightarrow 0$	Sí
RNN	D pasos	$\ \nabla\ \leq \omega^D \rightarrow 0$	No
CNN	D/K pasos	$\ \nabla\ \leq \omega^{M-1} \rightarrow 0$	Sí
Transformer	1 paso	$\ \nabla\ \leq C$	Sí

Cuadro 2.1: Comparativa de las arquitecturas estudiadas. D : distancia entre tokens, K : tamaño del kernel, M : número de capas, C : constante independiente de la profundidad.

2.6. De Transformers a LLMs: el Modelo GPT

Una vez establecidos los fundamentos teóricos de la arquitectura, esta se especializa para la generación de lenguaje natural, dando lugar a los denominados Grandes Modelos de Lenguaje (LLMs, *Large Language Models*).

El **modelo GPT** (*Generative Pre-trained Transformer*), propuesto por Radford et al. [9], utiliza exclusivamente el decodificador del Transformer descrito en la Sección 2.2, descartando el codificador y la atención cruzada. Cada bloque consta de dos sub-capas: una de autoatención enmascarada (Ecuación 2.7), que impide que cada token atienda a posiciones futuras, y una red *feed-forward*, estando ambas envueltas por conexiones residuales y normalización de capa tal y como se analizó en la Sección 2.5.

2.6.1. Objetivo de Entrenamiento

El entrenamiento de GPT se basa en la predicción del siguiente token basándose exclusivamente en los anteriores. Dada una secuencia de tokens $\mathbf{x} = (x_1, x_2, \dots, x_n)$, en cada posición i la salida del último bloque del decodificador (un vector $\mathbf{h}_i \in \mathbb{R}^{d_{model}}$) se multiplica por una matriz de pesos $W_{vocab} \in \mathbb{R}^{d_{model} \times |V|}$, produciendo un vector de puntuaciones $\mathbf{z}_i \in \mathbb{R}^{|V|}$ sobre todo el vocabulario V , al que se aplica la función *softmax* para obtener la probabilidad de cada token:

$$P(x_i | x_1, \dots, x_{i-1}; \theta) = \text{softmax}(\mathbf{z}_i)_{x_i} \quad (2.20)$$

donde θ denota los parámetros del modelo. La función de pérdida es la negativa de la log-verosimilitud, sumada sobre todas las posiciones:

$$\mathcal{L}(\mathbf{x}) = - \sum_{i=1}^n \log P(x_i | x_1, \dots, x_{i-1}; \theta) \geq 0 \quad (2.21)$$

La autoatención enmascarada garantiza que cada $P(x_i | x_1, \dots, x_{i-1})$ dependa únicamente de los tokens previos, preservando la propiedad auto-regresiva.

2.6.2. Preentrenamiento y Ajuste Fino

El proceso de aprendizaje de GPT se divide en dos fases:

- **Preentrenamiento:** El modelo se entrena con grandes volúmenes de texto sin etiquetar, optimizando la Ecuación (2.21). Durante esta fase, el modelo aprende representaciones generales del lenguaje: gramática, semántica y capacidad de razonamiento.
- **Ajuste fino (*fine-tuning*):** Una vez preentrenado, el modelo se adapta a una tarea específica reentrenándolo con un *dataset* (conjunto de datos) pequeño y especializado, partiendo de los pesos ya aprendidos y ajustando sus parámetros para optimizar el rendimiento en la tarea concreta.

Este proceso de preentrenamiento seguido de ajuste fino es precisamente el que se emplea en el Capítulo 3 para la construcción del chatbot. La Tabla 2.2 resume la evolución de los modelos GPT.

Modelo	Año	Parámetros	Aportación principal
GPT	2018	117M	Preentrenamiento + ajuste fino
GPT-2	2019	1.500M	Generación sin ajuste fino, textos más largos y coherentes
GPT-3*	2020	175.000M	Aprendizaje en contexto, resolución de tareas sin reentrenar
GPT-4	2023	No publicado	Multimodalidad, razonamiento sobre texto e imágenes
GPT-5	2025	No publicado	Razonamiento unificado, menor tasa de alucinaciones

*Base de ChatGPT (2022), que supuso una revolución de la industria.

Cuadro 2.2: Evolución de los modelos GPT.

Capítulo 3

Construcción de un Chatbot sobre el Modelo Transformer

3.1. Planteamiento y Modelo Base

Como aplicación práctica de los fundamentos teóricos desarrollados en el Capítulo 2, se ha construido un chatbot capaz de responder preguntas sobre el propio Transformer. La elección de esta temática responde a un doble propósito: por un lado, cierra el vínculo entre la teoría formalizada y su aplicación; por otro, define un dominio acotado y técnicamente preciso, lo que facilita la evaluación de las respuestas del modelo.

El principal condicionante del proyecto es la limitación de recursos computacionales, que descarta la posibilidad de entrenar un modelo desde cero. Para superar esta limitación, se ha adoptado el paradigma de preentrenamiento seguido de ajuste fino descrito en la Sección 2.6: se parte de un modelo ya preentrenado que aporta un conocimiento general del idioma, y se le especializa mediante ajuste fino sobre un conjunto de datos específico del dominio.

Concretamente, se ha empleado el modelo DeepESP/gpt2-spanish, una variante de GPT-2 con 124 millones de parámetros, preentrenada sobre un amplio corpus de texto en castellano y disponible públicamente a través de la biblioteca *Hugging Face Transformers* (la implementación ha sido en Python). El ajuste fino se ha realizado en Google Colab con GPU T4.

3.2. Dataset y Entrenamiento

Construcción del dataset

El dataset se ha construido manualmente a partir de una fuente principal: los contenidos del presente trabajo, los cuales han sido complementados con los artículos citados en la bibliografía. Cada ejemplo consiste en un par pregunta-respuesta en castellano sobre conceptos del modelo Transformer.

Se han elaborado tres versiones del dataset. La primera versión contiene 150 pares que cubren los conceptos fundamentales del Capítulo 2. La segunda amplía a 312 pares, incorporando conceptos adicionales sobre conexiones residuales, normalización de capa y el modelo GPT. La tercera versión extiende el dataset a 480 pares, añadiendo reformulaciones de las mismas preguntas con redacciones diferentes para incentivar el aprendizaje de patrones generales frente a la memorización literal. Tanto el dataset como la implementación se pueden consultar en el repositorio Transformer-chatbot de GitHub: [10].

Cada ejemplo del dataset se formatea con tokens especiales que delimitan la pregunta y la respuesta:

`<pregunta> texto de la pregunta <respuesta> texto de la respuesta <fin>`

Durante la preparación de los datos, los tokens correspondientes a la pregunta se enmascaran en la función de pérdida (asignándoles el valor -100), de modo que el modelo solo aprende a generar la respuesta condicionada a la pregunta, sin dedicar capacidad a memorizar las preguntas.

Entrenamiento

El ajuste fino se ha realizado utilizando la clase `Trainer` de la biblioteca *Hugging Face Transformers*, que gestiona automáticamente la estabilidad numérica y el escalado de gradientes.

El proceso de entrenamiento ha sido iterativo: a lo largo de 7 experimentos se han ajustado los hiperparámetros, el tamaño del dataset y la estrategia de cálculo de la función de pérdida hasta obtener la configuración final. El detalle completo de este proceso se recoge en el Apéndice C, mientras que los *scripts* de la implementación del modelo se encuentran en el Apéndice F.

En las versiones finales del dataset (312 y 480 ejemplos), el entrenamiento durante 50 épocas produjo un loss final de 0,006, indicando una memorización casi completa del dataset. La Figura 3.1 muestra la evolución de la función de pérdida durante el entrenamiento con 480 ejemplos.



Figura 3.1: Evolución de la función de pérdida $\mathcal{L}$ durante el séptimo entrenamiento del chatbot (50 épocas, 480 ejemplos).

3.3. Resultados

Para evaluar el chatbot se han realizado 30 preguntas reformuladas, no presentes en el dataset de entrenamiento, sobre conceptos teóricos del trabajo. La Tabla 3.1 resume los resultados obtenidos con cuatro versiones del dataset, incluyendo la evaluación del modelo base mediante *zero-shot learning* (0 ejemplos).

	0 ej.	150 ej.	312 ej.	480 ej.
Correctas	0 (0 %)	4 (13 %)	8 (27 %)	19 (63 %)
Parciales	0 (0 %)	9 (30 %)	7 (23 %)	7 (23 %)
Incorrectas	30 (100 %)	17 (57 %)	15 (50 %)	4 (13 %)
Aceptables	0 %	43 %	50 %	87 %

Cuadro 3.1: Resultados en 30 preguntas reformuladas según el tamaño del dataset de ajuste fino. Salvo el modelo base (0 ejemplos), el resto de modelos fueron entrenados durante 50 épocas.

La progresión es clara: sin ajuste fino (0%), el modelo genera texto en español gramaticalmente correcto pero ajeno al dominio, lo que confirma que el preentrenamiento aporta el idioma pero no el conocimiento técnico. Con 150 y 312 ejemplos, el modelo adquiere vocabulario del dominio pero mezcla conceptos (43 % y 50 % de respuestas aceptables, respectivamente). El salto más significativo se produce al ampliar a 480 ejemplos con reformulaciones (87%), donde 12 de las 15 preguntas anteriormente

incorrectas pasan a responderse de forma adecuada. El siguiente par ilustra este contraste al definir el desvanecimiento del gradiente:

Con 312 ejemplos: Es una técnica que limita la norma del gradiente a un valor máximo antes de actualizar los pesos.

Con 480 ejemplos: Es el fenómeno donde la norma del gradiente decrece exponencialmente al retropropagarse a través de muchas capas. Se produce porque al aplicar la regla de la cadena, se multiplican secuencialmente jacobianas cuya norma es menor que 1, haciendo que el producto converja a cero.

Una selección más amplia de respuestas se recoge en el Apéndice C. Además, se ha realizado un estudio de la reproducibilidad de las respuestas, el cual se puede consultar en el Apéndice D.

Análisis

Los resultados revelan tres factores determinantes. Primero, el preentrenamiento aporta la estructura lingüística (todas las respuestas son español correcto) pero no el conocimiento específico del dominio. Segundo, el ajuste fino inyecta el vocabulario técnico y adapta al modelo al formato de pregunta-respuesta. Tercero, la diversidad del dataset es más determinante que su tamaño absoluto para garantizar la generalización: el gran salto de rendimiento del 50 % al 87 % se produce al añadir reformulaciones, no simplemente al añadir preguntas conceptualmente nuevas. De hecho, los modelos ajustados con 312 y 480 ejemplos alcanzan la misma pérdida final en el entrenamiento (0,006), pero el entrenado con mayor diversidad de formulaciones generaliza notablemente mejor ante preguntas (*prompts*) inéditas.

3.4. Conclusiones y Trabajo Futuro

El presente trabajo ha abordado la arquitectura Transformer desde una doble perspectiva: la formalización matemática de sus mecanismos fundamentales y su aplicación práctica en la construcción de un chatbot.

En el **plano teórico** se ha utilizado el desvanecimiento del gradiente como hilo conductor para desarrollar un análisis riguroso del mecanismo de autoatención, la codificación posicional y la estabilidad del gradiente, incluyendo demostraciones sobre la equivariancia a permutaciones, la propiedad de rotación posicional, el flujo del gradiente en conexiones residuales, y la diferencia entre Post-LN y Pre-LN.

En el **plano práctico**, la comparación entre los datasets de diferentes tamaños ha confirmado que la diversidad de los datos es más determinante que el valor de la función de pérdida para la generalización del modelo, pasando de un 50 % (312 ejemplos) a un 87 % (480 ejemplos) de respuestas aceptables en preguntas reformuladas.

Como **líneas de trabajo futuro** para mejorar el chatbot, se identifican las siguientes direcciones:

- Ampliar el dataset con un mayor volumen de reformulaciones para cubrir los conceptos que aún fallan.
- Emplear un modelo de menor tamaño, como DistilGPT-2, para forzar mayor generalización al reducir la capacidad de memorización.
- Aplicar técnicas de regularización como *early stopping* o *dropout* más agresivo durante el ajuste fino.

En conclusión, el campo de los modelos de lenguaje evoluciona a un ritmo vertiginoso: desde la publicación del Transformer en 2017, han surgido arquitecturas cada vez más grandes y sofisticadas que han transformado la Inteligencia Artificial. No obstante, todos estos modelos comparten los fundamentos formalizados en este trabajo: la autoatención, las conexiones residuales, la normalización de capa y el paradigma de preentrenamiento seguido de ajuste fino. Comprender estos mecanismos y sus propiedades matemáticas resulta esencial para abordar con rigor cualquier avance futuro en este campo.

Bibliografía

- [1] I. GOODFELLOW, Y. BENGIO Y A. COURVILLE, *Deep Learning*, MIT Press, Cambridge, 2016, disponible en <https://www.deeplearningbook.org/>.
- [2] Y. LECUN, Y. BENGIO Y G. HINTON, Deep learning, *Nature*, (2015), 436–444, <https://www.nature.com/articles/nature14539>.
- [3] Y. BENGIO, P. SIMARD Y P. FRASCONI, *Learning long-term dependencies with gradient descent is difficult*, IEEE transactions on neural networks **5** (2) (1994), 157–166, disponible en <https://ieeexplore.ieee.org/document/279181>.
- [4] G. CYBENKO, *Approximation by superpositions of a sigmoidal function*, Mathematics of Control, Signals and Systems **2** (4) (1989), 303–314, disponible en <https://doi.org/10.1007/BF02551274>.
- [5] A. VASWANI ET AL., Attention is all you need, *Advances in Neural Information Processing Systems (NeurIPS)*, (2017), 5998–6008, <https://arxiv.org/abs/1706.03762>.
- [6] K. HE, X. ZHANG, S. REN Y J. SUN, *Deep residual learning for image recognition*, Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 770–778 (2016), disponible en <https://arxiv.org/abs/1512.03385>.
- [7] J. L. BA, J. R. KIROS Y G. E. HINTON, *Layer Normalization*, arXiv preprint arXiv:1607.06450 (2016), disponible en <https://arxiv.org/abs/1607.06450>.
- [8] R. XIONG, Y. YANG, D. HE, K. ZHENG, S. ZHENG, C. XING, H. ZHANG, Y. LAN, L. WANG Y T.-Y. LIU, *On Layer Normalization in the Transformer Architecture*, arXiv preprint arXiv:2002.04745 (2020), disponible en <https://arxiv.org/abs/2002.04745>.
- [9] A. RADFORD, K. NARASIMHAN, T. SALIMANS Y I. SUTSKEVER, *Improving Language Understanding by Generative Pre-Training*, OpenAI (2018), disponible en https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.
- [10] V. SIERRA VICÉN, *Transformer-chatbot*, GitHub, 2026, disponible en <https://github.com/vsier56/Transformer-chatbot>.
- [11] P. J. WERBOS, *Backpropagation through time: what it does and how to do it*, Proceedings of the IEEE **78** (10) (1990), 1550–1560, disponible en <https://ieeexplore.ieee.org/document/58337>.

Apéndice A

Modelos RNN - CNN

Actualmente, las estructuras RNN-CNN han sido prácticamente desplazadas hacia el desuso en el Procesamiento del Lenguaje Natural. Sin embargo, es importante comprender su funcionamiento para poder avanzar hacia estructuras más complejas.

A.1. Redes Neuronales Recurrentes

Hemos visto en el capítulo anterior que las redes neuronales asumen independencia entre las entradas (como en el caso de las redes *feed-forward*). Sin embargo, hay casos en los que la información que se está tratando depende en gran parte de un contexto previo.

Las **Redes Neuronales Recurrentes** (RNN) están diseñadas específicamente para procesar datos secuenciales $\mathbf{x} = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n)$ y mantener una memoria de estados. Para lograrlo, se introduce un estado oculto $\mathbf{h}_t$, un vector que actúa como la memoria interna de la red y que se actualiza iterativamente a lo largo de la secuencia, combinando la entrada actual con la información que ya se tiene de iteraciones anteriores.

En cada iteración t la red posee una entrada $\mathbf{x}_t$, la cual combina con el estado oculto de la iteración anterior $\mathbf{h}_{t-1}$, para obtener un nuevo estado $\mathbf{h}_t$. Este proceso se puede describir como:

$$\mathbf{h}_t = \sigma(W_{xh}\mathbf{x}_t + W_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \quad (\text{A.1})$$

donde W_{xh} es la matriz de pesos asociada a la entrada en el instante t , W_{hh} es la matriz de transición que pondera el estado oculto del instante anterior $t - 1$, y $\mathbf{b}_h$ es el vector de sesgo. Además, estas matrices de pesos son compartidas a lo largo de toda la secuencia, lo que permite a la red procesar textos de longitud variable. Dicho de otra forma, lo único que evoluciona dinámicamente es el estado oculto.

Posteriormente, a partir del estado oculto $\mathbf{h}_t$ se puede calcular la salida $\hat{\mathbf{y}}_t$:

$$\hat{\mathbf{y}}_t = W_{hy}\mathbf{h}_t + \mathbf{b}_y \quad (\text{A.2})$$

donde W_{hy} es la matriz de pesos entre la capa oculta y la salida, y $\mathbf{b}_y$ su respectivo sesgo.

Para comprender este flujo de información, resulta útil visualizar la red de forma desenrollada a través del tiempo, tal y como se muestra en la Figura A.1. En el instante inicial $t = 1$, el estado oculto previo $\mathbf{h}_0$ suele inicializarse como un vector de ceros. A partir de ahí, la misma estructura de pesos se aplica iterativamente para cada elemento de la secuencia.

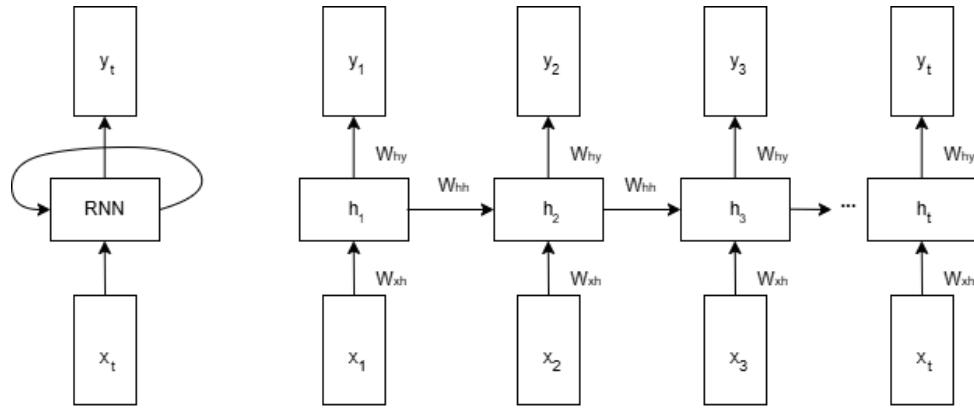


Figura A.1: Esquema de una RNN

Una vez obtenida la salida $\hat{y}_t$, suele proyectarse mediante una función *softmax* (explicada en Apéndice B) para obtener una distribución de probabilidad sobre todo el vocabulario disponible. La palabra con mayor probabilidad será la predicción de la red para ese instante de tiempo.

Para entrenar y optimizar los pesos de estas matrices compartidas (W_{xh} , W_{hh} , W_{hy}), es necesario definir una métrica de error.

Definición 9. La **función de pérdida global** $\mathcal{L}$ de una secuencia completa extiende la función de pérdida dada en la Definición 1, sumando las pérdidas locales evaluadas en cada instante t :

$$\mathcal{L}(\mathbf{y}, \hat{\mathbf{y}}) = \sum_{t=1}^n \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t) \quad (\text{A.3})$$

donde $\mathbf{y}_t$ es el valor real esperado, $\hat{\mathbf{y}}_t$ es la predicción del modelo, y $\mathcal{L}_t$ la función de entropía cruzada. Esta función mide la divergencia entre dos distribuciones de probabilidad: la real (que suele tener probabilidad 1 en la palabra correcta y 0 en las demás) y la predicha por la red. En términos prácticos, penaliza fuertemente al modelo cuando hace una predicción incorrecta con un alto nivel de confianza, obligando a la red a asignar la mayor probabilidad posible a la salida correcta.

Una vez definida la función de pérdida a lo largo de toda la secuencia, el siguiente paso es actualizar los parámetros de la red para minimizar dicho error. Es aquí donde la naturaleza cíclica de las RNN introduce severos desafíos matemáticos.

Limitaciones de las RNN: Dependencias a Largo Plazo

Como se ha explicado en la Sección 1.2.1, para actualizar los parámetros de la red y minimizar la función de pérdida $\mathcal{L}$ a lo largo de la secuencia, se emplea el algoritmo de retropropagación a través del tiempo [11]. Este método consiste en desplegar la red recurrente a lo largo de los n instantes de tiempo y aplicar la regla de la cadena convencional desde la salida final hasta los estados iniciales.

Sin embargo, esto supone un desafío analítico severo al procesar secuencias largas. Supongamos que queremos evaluar cómo afecta el estado oculto en un instante temprano k a la pérdida en un instante futuro t (donde $t \gg k$). Para ello, necesitamos calcular el gradiente $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$.

Dado que la red procesa la información de forma estrictamente secuencial, no existe una conexión directa entre k y t ; la señal de error debe atravesar todos los estados intermedios. Si analizamos un salto corto, por ejemplo para retroceder desde t hasta $t-2$, la regla de la cadena nos obliga a multiplicar las derivadas parciales de cada paso intermedio:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-2}} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdot \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{h}_{t-2}}$$

Al generalizar este proceso para un salto temporal largo desde el instante t hasta el instante k , debemos multiplicar secuencialmente todas las matrices jacobianas que conectan dichos instantes:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdot \frac{\partial \mathbf{h}_{t-1}}{\partial \mathbf{h}_{t-2}} \cdots \frac{\partial \mathbf{h}_{k+2}}{\partial \mathbf{h}_{k+1}} \cdot \frac{\partial \mathbf{h}_{k+1}}{\partial \mathbf{h}_k}$$

Por tanto, podemos expresarlo como:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \quad (\text{A.4})$$

Proposición A.1 (Jacobiana del estado oculto de una RNN). Sea el estado oculto de una RNN definido según la Ecuación (A.1). La matriz jacobiana de $\mathbf{h}_j$ respecto a $\mathbf{h}_{j-1}$ es:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(\sigma'(\mathbf{z}_j)) \cdot W_{hh} \quad (\text{A.5})$$

donde $\mathbf{z}_j = W_{xh}\mathbf{x}_j + W_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h$ es la pre-activación y $\text{diag}(\sigma'(\mathbf{z}_j))$ es la matriz diagonal cuya entrada (k,k) es la derivada de la función de activación evaluada en la k -ésima componente de $\mathbf{z}_j$.

Demostración. Partimos de $\mathbf{h}_j = \sigma(\mathbf{z}_j)$ con $\mathbf{z}_j = W_{xh}\mathbf{x}_j + W_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h$. Aplicando la regla de la cadena:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \frac{\partial \mathbf{h}_j}{\partial \mathbf{z}_j} \cdot \frac{\partial \mathbf{z}_j}{\partial \mathbf{h}_{j-1}}$$

Para el segundo factor, dado que $\mathbf{z}_j$ depende linealmente de $\mathbf{h}_{j-1}$ a través de W_{hh} (los términos $W_{xh}\mathbf{x}_j$ y $\mathbf{b}_h$ son constantes respecto a $\mathbf{h}_{j-1}$), se tiene:

$$\frac{\partial \mathbf{z}_j}{\partial \mathbf{h}_{j-1}} = W_{hh}$$

Para el primer factor, como la función de activación σ se aplica componente a componente, la derivada de la componente k -ésima de $\mathbf{h}_j$ solo depende de la componente k -ésima de $\mathbf{z}_j$:

$$\frac{\partial h_{j,k}}{\partial z_{j,m}} = \begin{cases} \sigma'(z_{j,k}) & \text{si } k = m \\ 0 & \text{si } k \neq m \end{cases}$$

lo que en forma matricial corresponde a $\frac{\partial \mathbf{h}_j}{\partial \mathbf{z}_j} = \text{diag}(\sigma'(\mathbf{z}_j))$. Sustituyendo ambos factores se obtiene el resultado. $\square$

Llegados a este punto, nos encontramos exactamente ante el mismo fenómeno de atenuación demostrado en el Capítulo 1. Como se ha probado en la Proposición A.1, cada matriz jacobiana del productorio tiene la forma $\text{diag}(\sigma'(\mathbf{z}_j)) \cdot W_{hh}$, donde la misma matriz W_{hh} aparece en todos los factores. Dado que para funciones de activación como la sigmoide se cumple $\sigma'(z) \leq 0,25$ para todo z , la norma de estas jacobianas es sistemáticamente menor que 1, y el producto iterativo tenderá a cero a medida que la distancia temporal $(t - k)$ aumente.

En el contexto del Procesamiento del Lenguaje Natural, esto significa que la RNN no puede aprender dependencias sintácticas o semánticas a largo plazo (por ejemplo, relacionar el sujeto al principio de un párrafo con su verbo al final). Es esta limitación estructural la que ha provocado su desplazamiento casi total por arquitecturas más modernas.

A.2. Redes Neuronales Convolucionales

Las **Redes Neuronales Convolucionales** (CNN) son un tipo de redes neuronales diseñadas para procesar datos que presentan una topología de cuadrícula regular. Su característica fundamental es que, en al menos una de sus capas, sustituyen las conexiones densas tradicionales por la operación matemática de convolución.

Aunque históricamente se popularizaron en la visión artificial para modelar la estructura espacial de las imágenes (2D), posteriormente se aplicaron en el Procesamiento del Lenguaje Natural sobre secuencias unidimensionales (1D).

Operación Convolución

Para comprender la arquitectura de una Red Neuronal Convolutiva, es imprescindible definir la operación lineal que le da nombre. La convolución es una operación lineal que describe la superposición de dos funciones al combinarlas.

Definición 10. Sean $x, w : \mathbb{R} \rightarrow \mathbb{R}$, la **operación convolución** se define como:

$$s(t) := (x * w)(t) = \int_{-\infty}^{\infty} x(\tau)w(t - \tau)d\tau$$

Como al trabajar con redes neuronales convolucionales generalmente el tiempo está discretizado, t solo tomará valores enteros ($t \in \mathbb{Z}_{\geq 0}$). En el dominio discreto, denotemos por X al espacio de las señales de entrada y por W al espacio de los filtros o *kernels*, los cuales se tratan de unidades parametrizables que se encargan de extraer características locales.

Definición 11. Sean $\mathbf{x} : \mathbb{Z} \rightarrow \mathbb{R}$ una señal discreta y $\mathbf{w} : \mathbb{Z} \rightarrow \mathbb{R}$ un filtro con soporte finito. Definimos la **operación de convolución unidimensional** (1D) como la aplicación s :

$$s : X \times W \rightarrow S$$

$$(\mathbf{x}, \mathbf{w}) \mapsto s(t) = (\mathbf{x} * \mathbf{w})(t) = \sum_{a=-\infty}^{\infty} \mathbf{x}(a)\mathbf{w}(t - a) \quad (\text{A.6})$$

Por la propia naturaleza operativa de la convolución descrita, una capa que emplea un *kernel* de tamaño K solo puede procesar y relacionar K elementos adyacentes a la vez. Esta limitación local es precisamente lo que dota a las CNN de su gran eficiencia computacional frente a los modelos clásicos.

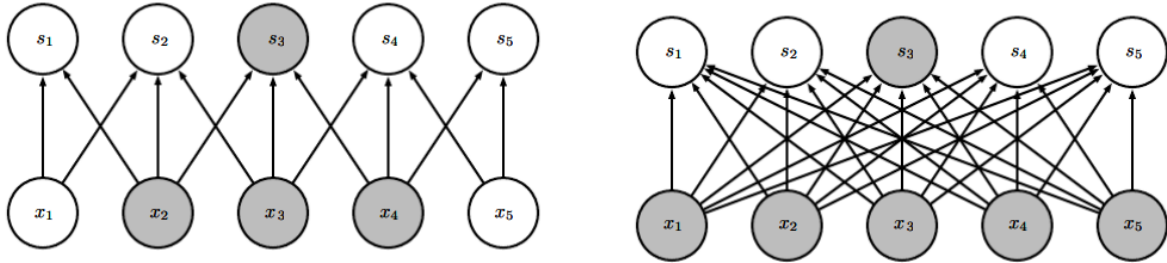


Figura A.2: Comparativa de conectividad arquitectónica en una CNN.

Si observamos la Figura A.2, en una red densa o completamente conectada (parte derecha), cada unidad de salida interactúa con todas las unidades de entrada mediante multiplicación matricial, lo que implica una complejidad paramétrica de $\mathcal{O}(N^2)$ si asumimos N entradas y N salidas. Por el contrario, como se observa en la parte izquierda, al aplicar un filtro de tamaño K (en este caso, $K = 3$), la unidad de salida s_3 únicamente está conectada a su entorno receptivo más inmediato (x_2, x_3 y x_4). Al reducir el tamaño del kernel a $K \ll N$, la complejidad paramétrica desciende a $\mathcal{O}(K \times N)$.

Esta formulación unidimensional (Ecuación A.6) es la que se aplica sobre secuencias de texto en el Procesamiento del Lenguaje Natural. A modo comparativo, cuando el dominio de entrada corresponde a datos con una estructura bidimensional, como una imagen I , el filtro $\mathbf{w}$ también adquiere dos dimensiones espaciales, generalizándose la operación de la siguiente manera:

$$S(i, j) = (I * \mathbf{w})(i, j) = \sum_a \sum_b I(a, b) \cdot \mathbf{w}(i - a, j - b)$$

A.2.1. Estructura de las CNN

La arquitectura de una Red Neuronal Convolutiva se estructura conceptualmente en tres bloques principales, a través de los cuales la información fluye desde su representación en bruto hasta la predicción final:

1. **Capa de Entrada:** En el procesamiento de texto, esta capa se encarga de transformar la secuencia discreta de palabras en una representación numérica continua. Dado un texto de N palabras, cada término se mapea a un vector o *embedding* de dimensión d . De este modo, la entrada a la red se define como una matriz $\mathbf{X} \in \mathbb{R}^{N \times d}$, donde cada fila representa las características semánticas de una palabra en la secuencia temporal.
2. **Aprendizaje de Características:** Esta es la etapa central de la arquitectura, cuyo objetivo es extraer patrones de la matriz de entrada $\mathbf{X}$ mediante la aplicación sistemática de tres operaciones:
 - Primero, en la **Capa de Convolución**, se aplican *kernels* (Ecuación A.6) para generar mapas de pre-activaciones que detectan patrones locales.
 - A continuación, en la **Capa de Activación**, el mapa resultante se procesa mediante una función no lineal (típicamente ReLU, $h(t) = \max(0, s(t))$) para preservar únicamente las señales relevantes.
 - Finalmente, en la **Capa de Agrupamiento**, se reducen las dimensiones espaciales (o en el caso 1D las longitudes de secuencia) de los mapas de activación, habitualmente mediante el llamado *Max-Pooling*, con el objetivo de resumir la información en la más importante y disminuir la complejidad computacional.
3. **Clasificación:** Los mapas de activación resultantes se “aplanan” y se introducen en capas completamente conectadas (*fully connected layers*). Estas capas actúan como un clasificador tradicional, obteniendo el resultado final de la red.

Limitaciones de las CNN: Campo Receptivo

A pesar de que las Redes Neuronales Convolutivas resuelven en gran medida el problema de la secuencialidad estricta de las RNN (permitiendo la paralelización del cálculo), presentan una debilidad estructural significativa cuando se aplican a textos largos: la limitación de su campo receptivo.

El **campo receptivo** se define como la región del espacio de entrada (en el Procesamiento del Lenguaje Natural, el número de palabras consecutivas) que influye en el cálculo de una activación concreta en una capa dada de la red.

Como se ha establecido en la definición de la operación, un filtro convolutivo está restringido a procesar únicamente K elementos adyacentes a la vez. Por lo tanto, si la red necesita capturar una dependencia semántica entre dos palabras que están separadas por una distancia D (donde $D \gg K$), una única capa convolutiva es estructuralmente incapaz de relacionarlas de forma directa.

Para solucionar este problema y lograr que la red asocie palabras lejanas, la arquitectura debe forzar el crecimiento del campo receptivo apilando múltiples capas de convolución y agrupamiento. Tras apilar M capas convolutivas estándar con un filtro de tamaño K , el campo receptivo efectivo R_M de las neuronas en la última capa crece de forma lineal:

$$R_M = M(K - 1) + 1 \quad (\text{A.7})$$

Proposición A.2 (Campo receptivo de M capas convolutivas). Sea una red de M capas convolutivas estándar, cada una con un filtro de tamaño K y paso (*stride*) igual a 1. El campo receptivo R_M de las neuronas en la última capa satisface:

$$R_M = M(K - 1) + 1 \quad (\text{A.8})$$

Demostración. Procedemos por inducción sobre el número de capas M .

- **Caso base** ($M = 1$): Una única capa convolucional con filtro de tamaño K conecta cada neurona de salida con exactamente K elementos consecutivos de la entrada. Por tanto, $R_1 = K = 1 \cdot (K - 1) + 1$.
- **Paso inductivo:** Supongamos que tras M capas el campo receptivo es $R_M = M(K - 1) + 1$, es decir, cada neurona de la capa M depende de R_M elementos consecutivos de la entrada original.

Al añadir una capa $M + 1$ con filtro de tamaño K y *stride* 1, cada neurona de la nueva capa observa K neuronas consecutivas de la capa anterior (consecutivas por la hipótesis de *stride* 1). Denotemos estas neuronas por las posiciones $p, p + 1, \dots, p + K - 1$ de la capa M .

Cada una de estas K neuronas tiene un campo receptivo de R_M sobre la entrada original. La neurona en la posición p cubre las posiciones de la entrada desde algún punto a hasta $a + R_M - 1$. La neurona en la posición $p + K - 1$ (la última de las K), al estar desplazada $K - 1$ posiciones, cubre desde $a + K - 1$ hasta $a + K - 1 + R_M - 1$.

El campo receptivo total de la neurona de la capa $M + 1$ es la unión de los campos receptivos de las K neuronas que observa, es decir, desde la posición a hasta la posición $a + K - 1 + R_M - 1$. Su tamaño es:

$$R_{M+1} = (a + K - 1 + R_M - 1) - a + 1 = R_M + (K - 1)$$

Aplicando la hipótesis de inducción:

$$R_{M+1} = M(K - 1) + 1 + (K - 1) = (M + 1)(K - 1) + 1$$

lo que completa la inducción.

□

En consecuencia, para modelar dependencias a largo plazo en un documento extenso, la red debe garantizar que $R_M \geq D$, lo que exige que el número de capas convolucionales M sea de gran tamaño, tal y como se ve en el siguiente Ejemplo.

Ejemplo 4. Supongamos que tenemos un párrafo de 1000 palabras que queremos que nuestro modelo entienda usando filtros de tamaño $K = 3$. En nuestro caso:

- $R_M = 1000$
- $K = 3$

Así, $1000 = M(3 - 1) + 1 \implies M \approx 500$. Por tanto, necesitaremos aproximadamente 500 capas convolucionales seguidas para procesar el texto.

Es exactamente en este punto donde la arquitectura choca con el problema del desvanecimiento del gradiente. Durante la fase de aprendizaje, la señal de error se calcula en la etapa de clasificación final y se retropropaga hacia atrás. Sin embargo, al tener que atravesar una cantidad M tan profunda de capas convolucionales, las multiplicaciones sucesivas hacen que la norma del gradiente se atenúe severamente.

Tal y como se demostró en el Capítulo 1, ocurre que si $\omega < 1$, se cumple que $\lim_{M \rightarrow \infty} \omega^{M-1} = 0$. Esto provoca que el gradiente colapse antes de llegar a las primeras capas de la etapa de extracción de características. Al recibir un gradiente nulo, estas capas convolucionales iniciales dejan de aprender, volviendo al modelo incapaz de comprender el texto desde su base.

Visión Artificial frente a Lenguaje Natural

En conclusión, es vital destacar una distinción crucial respecto a la naturaleza de los datos. Las arquitecturas CNN son extraordinariamente eficaces en visión artificial porque, en una imagen bidimensional, las características relevantes (bordes, texturas o partes de un objeto) están fuertemente localizadas. Un *kernel* pequeño que observe una subcuadrícula de 2×2 es capaz de capturar un patrón completo (como un ojo) porque todos los píxeles que lo conforman son espacialmente contiguos, tal y como se observa en la Figura A.3.

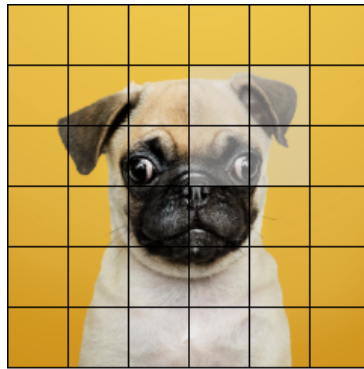


Figura A.3: Distribución espacial de una imagen. Kernel 2×2 aplicado sobre la zona sombreada para localizar el ojo izquierdo.

Sin embargo, el lenguaje humano está repleto de dependencias semánticas a larga distancia, como es el caso de “mujer” y “marido” en la Figura A.4, donde la red necesita recordar el sujeto que inició la acción una decena de palabras atrás para comprender el sentido final de la oración.



Figura A.4: Distribución espacial de una cadena de texto

El hecho de que las CNN necesiten una profundidad extrema para lograr conectar conceptos separados en una oración demuestra que no son herramientas óptimas para procesar textos largos. Esta limitación estructural compartida, tanto por las RNN como por las CNN, es precisamente la que motiva la introducción del Transformer en el Capítulo 2.

Apéndice B

Función Softmax

Definición 12. Sea $\mathbf{z} = (z_1, \dots, z_n) \in \mathbb{R}^n$ un vector de puntuaciones reales. La función **softmax** se define componente a componente como:

$$\text{softmax} : \mathbb{R}^n \rightarrow (0, 1)^n$$
$$\mathbf{z} \mapsto \text{softmax}(\mathbf{z}) = \left(\frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \right)_{i=1}^n \quad (\text{B.1})$$

Esta función verifica de forma inmediata las siguientes propiedades:

1. **Positividad:** Dado que $e^{z_i} > 0$ para todo $z_i \in \mathbb{R}$, se cumple que $\text{softmax}(\mathbf{z})_i > 0$ para todo i .
2. **Normalización:** Por construcción del denominador,

$$\sum_{i=1}^n \text{softmax}(\mathbf{z})_i = \sum_{i=1}^n \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} = \frac{\sum_{i=1}^n e^{z_i}}{\sum_{j=1}^n e^{z_j}} = 1$$

Estas dos propiedades permiten interpretar la salida de la función *softmax* como una distribución de probabilidad discreta sobre n categorías (las componentes del vector $\mathbf{z}$). En el contexto de la función de atención (Ecuación 2.3), cada fila de la matriz $QK^T / \sqrt{d_k}$ contiene las puntuaciones de afinidad de un token con todos los demás. La función *softmax*, aplicada fila por fila, transforma dichas puntuaciones en los pesos de atención que determinan la suma ponderada de los valores.

Amplificación de diferencias

La función exponencial aumenta las diferencias entre las componentes del vector de entrada. Si $z_i > z_j$, entonces $e^{z_i} \gg e^{z_j}$ conforme crece la diferencia $z_i - z_j$. Esto provoca que la distribución resultante se concentre en las componentes con mayor puntuación, lo que permite al modelo focalizar su atención en los tokens más relevantes.

En el límite, cuando una componente z_k domina al resto ($z_k \gg z_j$ para todo $j \neq k$), *softmax* tiende a una distribución degenerada:

$$\lim_{z_k \rightarrow \infty} \text{softmax}(\mathbf{z})_i = \begin{cases} 1 & \text{si } i = k \\ 0 & \text{si } i \neq k \end{cases}$$

Justificación del factor de escala $1/\sqrt{d_k}$

Como se demuestra en la Proposición 2.1, cuando la dimensión d_k es elevada, los productos escalares $\mathbf{q} \cdot \mathbf{k}$ tienen varianza d_k , lo que empuja a la *softmax* a la región saturada descrita anteriormente. Dividir por $\sqrt{d_k}$ normaliza la varianza a 1, manteniendo las puntuaciones en la zona operativa de la función.

Apéndice C

Desarrollo y Evaluación del Chatbot

En este Apéndice se recoge la evolución de los experimentos realizados durante el ajuste fino del modelo, incluyendo los hiperparámetros empleados en cada iteración, las modificaciones realizadas y las funciones de pérdida resultantes.

Configuración base

Todos los experimentos parten del modelo DeepESP/gpt2-spanish (124M parámetros), ajustado sobre un dataset de pares pregunta-respuesta mediante la biblioteca *Hugging Face Transformers*. Para el entrenamiento se ha utilizado GPU T4 en Google Colab.

Hiperparámetros

A continuación se describe brevemente cada uno de los hiperparámetros configurados para el ajuste fino:

- `num_train_epochs`: Número de épocas, es decir, cuántas veces recorre el modelo el dataset completo durante el entrenamiento. Con 480 ejemplos y 50 épocas, el modelo ve cada ejemplo 50 veces.
- `per_device_train_batch_size`: Tamaño del *batch*. Determina cuántos ejemplos procesa el modelo simultáneamente antes de actualizar los pesos. Un *batch* mayor estabiliza el entrenamiento pero requiere más memoria.
- `learning_rate`: Tasa de aprendizaje η , el escalar que controla la magnitud de las actualizaciones de pesos en cada paso (Sección 1.2.1).
- `warmup_steps`: Número de pasos iniciales durante los cuales el *learning rate* aumenta progresivamente desde un valor cercano a cero hasta su valor objetivo. Evita actualizaciones bruscas al inicio del entrenamiento, cuando los gradientes pueden ser inestables.
- `weight_decay`: Coeficiente de regularización que penaliza los pesos grandes durante el entrenamiento, añadiendo un término proporcional a $\|\theta\|^2$ en la función de pérdida. Incentiva soluciones más simples y reduce el riesgo de *overfitting*.
- `max_grad_norm`: Umbral de *gradient clipping*. Si la norma del gradiente supera este valor, se reescala proporcionalmente para prevenir la explosión del gradiente.
- `fp16`: Precisión mixta de 16 bits. Cuando hay GPU disponible, los cálculos se realizan en formato *float16* en lugar de *float32*, duplicando la velocidad y reduciendo el consumo de memoria.

- **seed**: Semilla aleatoria que fija la inicialización de los pesos y el orden de los datos, garantizando que el experimento sea reproducible.

La configuración de dichos hiperparámetros durante los entrenamientos ha sido la siguiente:

Exp.	Ejemplos	Épocas	LR	Warmup	W. Decay	Loss	Tiempo	Resultado
1	134	15	5×10^{-5}	50	0,01	NaN	—	Inestabilidad numérica
2	134	15	2×10^{-5}	30	0,01	1,50	~15 min	Respuestas incoherentes
3	231	50	2×10^{-5}	30	0,01	0,011	~25 min	Parcial (3/5 correctas)
4	256	50	2×10^{-5}	30	0,01	0,006	~30 min	<i>Overfitting</i>
5	312	25	2×10^{-5}	30	0,05	0,098	~20 min	Insuficiente
6	340	50	2×10^{-5}	30	0,05	0,006	~35 min	<i>Overfitting</i>
7	480	50	3×10^{-5}	50	0,05	0,006	~40 min	87% aceptable

Parámetros constantes: *batch size* = 4, *gradient clipping* = 0,5, fp16 (GPU), *seed* = 42.

Cuadro C.1: Resumen de los experimentos de ajuste fino realizados durante el desarrollo del chatbot.

Iteraciones de entrenamiento

En total se han realizado 7 experimentos, en los que se han probado distintas configuraciones de hiperparámetros y se ha evaluado el rendimiento del modelo tras cada modificación.

- **Experimento 1:** El entrenamiento con bucle manual produce valores NaN desde el segundo *batch* por inestabilidad numérica. Se ha migrado al *Trainer* de Hugging Face, que gestiona automáticamente la precisión y el escalado de gradientes.
- **Experimento 2:** Primera ejecución estable. La pérdida ha descendido a 1,50 pero las respuestas son incoherentes: el modelo mezcla conceptos sin estructura. Se ha identificado que el modelo aprende a predecir tanto la pregunta como la respuesta.
- **Experimento 3:** Se han enmascarado los tokens de la pregunta en la función de pérdida para que el modelo solo aprenda a generar respuestas. Se ha ampliado el dataset a 231 ejemplos. Las respuestas han mejorado significativamente (3 de 5 correctas).
- **Experimento 4:** Se ha ampliado a 256 ejemplos con refuerzo en autoatención y conexiones residuales. El *loss* ha descendido a 0,006, indicando memorización. Las preguntas del dataset se responden perfectamente pero las reformulaciones fallan.
- **Experimento 5:** Se han reducido las *epochs* a 25 y se ha aumentado el *weight decay* a 0,05 para combatir el *overfitting*. El *loss* de 0,098 ha resultado insuficiente: las respuestas han empeorado respecto al experimento anterior.
- **Experimento 6:** Se han restaurado las 50 *epochs* manteniendo el *weight decay* de 0,05, con 340 ejemplos. La pérdida ha vuelto a 0,006 con resultados similares al experimento 4: buena memorización pero escasa generalización.
- **Experimento 7:** Se ha ampliado el dataset a 480 ejemplos, siendo los nuevos únicamente reformulaciones. La pérdida se mantiene en 0,006, pero el modelo generaliza notablemente mejor: un 87% de respuestas aceptables en preguntas reformuladas.

Una vez obtenida la configuración final (Experimento 7), se han entrenado dos versiones adicionales del dataset (150 y 312 ejemplos) con los mismos hiperparámetros, junto con una evaluación del modelo base sin ajuste fino (0 ejemplos) para estudiar el efecto del tamaño y la diversidad del dataset sobre la generalización. Los resultados de esta comparación se presentan en la Sección 3.3.

Curvas de pérdida

A continuación se presenta la evolución de las funciones de pérdida $\mathcal{L}$ durante los últimos 4 experimentos.

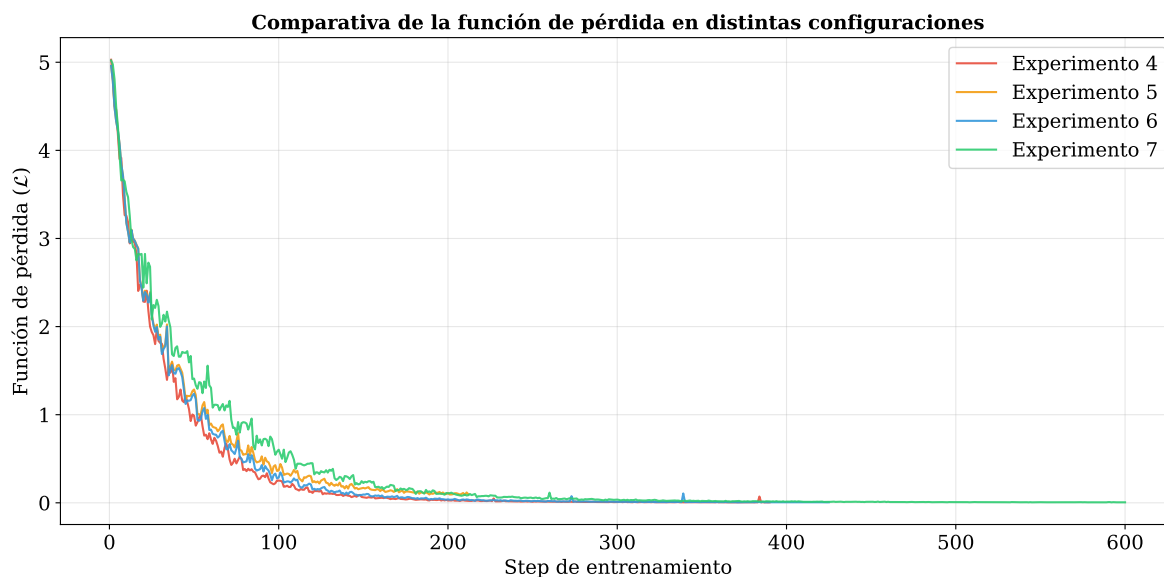


Figura C.1: Comparativa de las funciones de pérdida.

Para apreciar con mayor detalle las diferencias en la fase inicial del entrenamiento, la Figura C.2 muestra los primeros 300 steps de las mismas curvas.

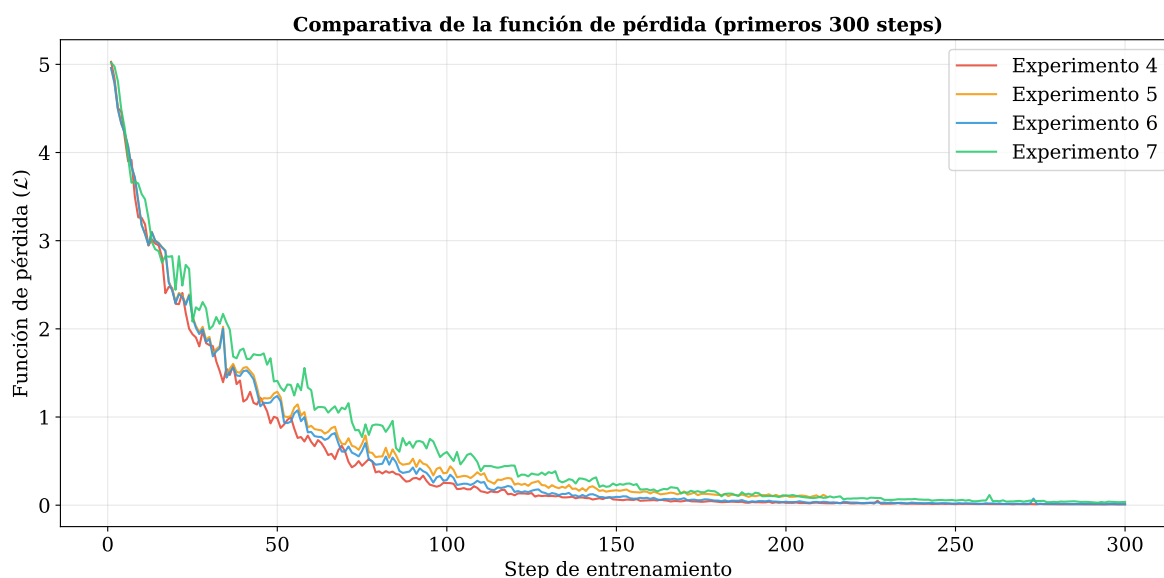


Figura C.2: Comparativa de las funciones de pérdida en los primeros 300 steps.

Evaluación con preguntas reformuladas

A continuación se presentan ejemplos representativos de las respuestas del chatbot (versión de 480 ejemplos) a preguntas no incluidas en el dataset de entrenamiento.

Respuestas correctas

Usuario: ¿Para qué sirven las claves en la atención?

Bot: Las claves representan lo que cada token ofrece para ser encontrado por otros. Se comparan con las consultas mediante producto escalar para determinar cuánta atención recibe cada token. Una alta compatibilidad entre consulta y clave indica que ese token contiene información relevante.

Usuario: ¿Qué es el vanishing gradient?

Bot: Es el fenómeno donde la norma del gradiente decrece exponencialmente al retropropagarse a través de muchas capas. Se produce porque al aplicar la regla de la cadena, se multiplican secuencialmente jacobianas cuya norma es menor que 1, haciendo que el producto converja a cero.

Usuario: ¿Cómo predice GPT la siguiente palabra?

Bot: En cada paso, la secuencia generada hasta el momento pasa por todos los bloques del decodificador. La salida del último bloque se proyecta mediante una capa lineal a un vector de puntuaciones sobre todo el vocabulario, y se aplica softmax para obtener la probabilidad de cada token. Se elige un token y se repite el proceso.

Usuario: ¿Cómo sabe el modelo qué información poner en Q, K y V?

Bot: Lo aprende durante el entrenamiento. Las matrices W^Q , W^K y W^V se inicializan aleatoriamente y se ajustan mediante retropropagación para minimizar la función de pérdida. El modelo descubre automáticamente qué proyecciones producen patrones de atención útiles para la tarea.

Respuestas parcialmente correctas

Usuario: ¿Qué diferencia hay entre autoatención y atención cruzada?

Bot: La autoatención permite que cada token atienda directamente al de forma directa, independientemente de su distancia. Además, la salida del último bloque se proyecta mediante una capa lineal seguida de softmax para obtener los pesos de atención.

Usuario: ¿Por qué el decodificador necesita una máscara?

Bot: Porque el codificador recibe la secuencia de entrada completa desde el principio y no genera nada. Su función es construir representaciones ricas de la salida, para lo cual cada token debe poder atender a todos los demás sin restricción. La máscara solo es necesaria en el decodificador, que genera tokens uno a uno y no debe ver el futuro.

Usuario: ¿Qué ventaja tiene Pre-LN sobre Post-LN?

Bot: La normalización se aplica después de la suma residual, envolviendo todo el bloque. En Post-LN, la jacobiana de la normalización multiplica a todo el producto incluido el término I. Esto permite que los gradientes fluyan sin atenuarse por las capas anteriores o iguales, impidiendo que sus derivadas colapsen y desestabilizarren¹ su entrenamiento.

Usuario: ¿Qué pasa si la softmax se satura?

Bot: La softmax recibe valores de gran magnitud, produciendo un producto escalar que permite interpretarlos como porcentajes de atención. En esta zona, los gradientes de la softmax son extremadamente pequeños, lo que dificulta el aprendizaje porque las actualizaciones de pesos son minúsculas.

¹Error léxico generado literalmente por el modelo de lenguaje. Se transcribe de forma exacta para ilustrar el comportamiento del chatbot.

Respuestas incorrectas

Usuario: ¿Por qué el Transformer es mejor que las RNN?

Bot: Porque tiene millones de parámetros que necesitan datos suficientes para encontrar buenas representaciones del lenguaje. El Vision Transformer divide imágenes en parches cuadrados que se tratan como tokens.

Usuario: ¿Qué diferencia hay entre preentrenar y hacer fine-tuning?

Bot: El modelo lee millones de documentos de texto y aprende a predecir la siguiente palabra en cada posición. Sin supervisión humana directa, adquiere gramática, vocabulario e información semántica. Esta fase es la más costosa y produce el conocimiento general que después se especializa con ajuste fino.

Usuario: ¿Qué es la función de pérdida en GPT?

Bot: Es una función que mide cuánto se equivoca el modelo al predecir cada token. Se calcula como la exponencial de la negativa de la log-verosimilitud: $x^{(l)} = x^{(l-1)} + F(x^{(l-1)})$. En lugar de aprender toda la transformación, el modelo aprende a minimizar esta función ajustando los parámetros del entrenamiento.

Usuario: ¿Por qué GPT no necesita codificador?

Bot: Porque GPT es un modelo generativo que solo debe encontrar una secuencia de entrada en cada paso del entrenamiento. No tiene una secuencia de salida separada que codificar como en traducción. Todo el proceso se repite hasta generar la respuesta token por token.

Comparación de resultados

Las Figuras C.3 y C.4 muestran mediante gráficos de sectores la evolución de los resultados en las 30 preguntas reformuladas para las cuatro versiones del dataset evaluadas.

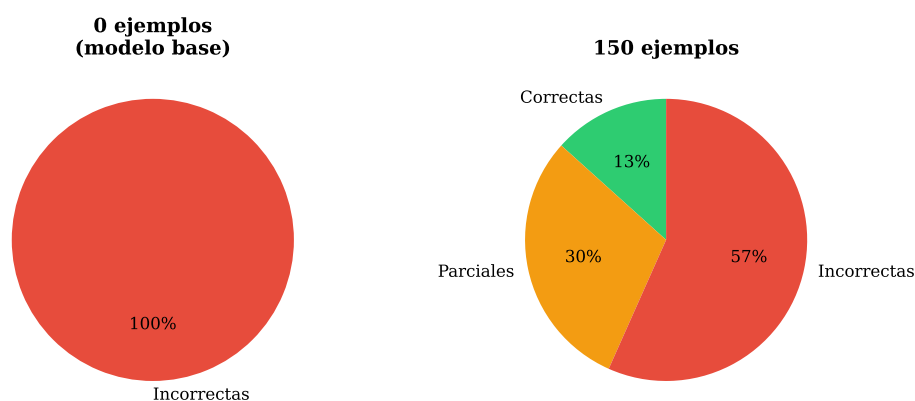


Figura C.3: Resultados con 0 ejemplos (modelo base) y 150 ejemplos.

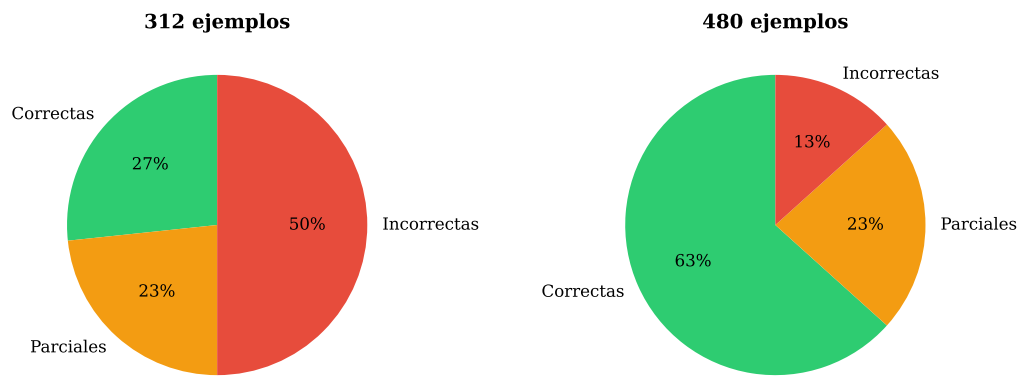


Figura C.4: Resultados con 312 y 480 ejemplos.

Apéndice D

Reproducibilidad de las Respuestas

Para estudiar el determinismo del chatbot, se ha formulado la misma pregunta tres veces consecutivas con dos configuraciones de temperatura, utilizando el mismo modelo entrenado.

Configuración

El parámetro *temperature* (T) controla la estocasticidad del modelo (la “creatividad”) durante la generación de texto. Antes de aplicar la función *softmax* para obtener la distribución de probabilidad del siguiente token, las puntuaciones $\mathbf{z}_i$ se dividen por este escalar T :

$$P(x_i) = \text{softmax} \left(\frac{\mathbf{z}_i}{T} \right)_{x_i}$$

Desde un punto de vista analítico, evaluar los límites de este hiperparámetro transforma por completo el comportamiento del modelo. Cuando $T = 1$, se recupera la distribución original aprendida. Cuando $T \rightarrow 0$, el cociente $\mathbf{z}_i/T$ amplifica las diferencias entre las componentes, provocando que la función *softmax* colapse. Así, el token con la puntuación máxima recibe una probabilidad absoluta de 1, y el resto de tokens una probabilidad de 0, produciendo una salida estrictamente determinista. Para este experimento, se han seleccionado 8 preguntas y cada una se ha repetido tres veces con:

- $T = 0,01$: configuración cuasi-determinista.
- $T = 0,3$: configuración estocástica habitual del chatbot.

Resultados

Pregunta	Temp. 0,3	Temp. 0,01
¿Qué es la autoatención?	2/3 idénticas	3/3 idénticas
¿Por qué se divide por $\sqrt{d_k}$?	3/3 idénticas	3/3 idénticas
¿Qué problema resuelve el Transformer?	3/3 idénticas	3/3 idénticas
¿Para qué sirve la normalización?	3/3 idénticas	3/3 idénticas
¿Cómo genera texto un chatbot GPT?	3/3 idénticas	3/3 idénticas
¿Qué son las conexiones residuales?	3/3 diferentes	2/3 idénticas
¿Cómo funciona la atención?	3/3 diferentes	3/3 idénticas
¿Por qué puede procesar en paralelo?	3/3 diferentes	3/3 idénticas

Cuadro D.1: Reproducibilidad de las respuestas según la temperatura de generación.

Análisis

- $T = 0,01$: las respuestas son idénticas en prácticamente todos los casos: el modelo elige siempre el token con mayor probabilidad, produciendo una salida determinista.
- $T = 0,3$: se observan dos comportamientos diferenciados. En 4 de las 8 preguntas, las respuestas son idénticas en las tres repeticiones, mientras que en una de ellas solo una respuesta varía. En las 3 restantes (conexiones residuales, funcionamiento de la atención y paralelización), las respuestas varían en cada ejecución.

Esta diferencia no se explica por la temperatura en sí, sino por el grado de confianza del modelo en cada concepto. Cuando el modelo ha aprendido un concepto con seguridad, la distribución de probabilidad que produce la *softmax* está tan concentrada en los tokens correctos que siempre produce la misma secuencia, incluso con $T = 0,3$. Cuando el modelo es inseguro, la distribución es más dispersa y se generan respuestas distintas.

Como ejemplo, ante la pregunta “¿Cómo funciona la atención en el Transformer?” ($T = 0,3$), las tres ejecuciones producen respuestas diferentes:

Ejecución 1: De forma auto-regresiva, token por token. En cada paso la consulta representa lo que ese token busca para ser encontrado por otros. La representación contextualizada de cada token es una suma ponderada de los vectores de valor del modelo.

Ejecución 2: De forma auto-regresiva, token por token. En cada paso la consulta representa lo que ese token busca en el contexto: los datos sirven para calcular cuánta importancia da cada palabra a cada una de las demás, y se multiplican por esa distribución mediante la codificación posicional.

Ejecución 3: De forma auto-regresiva, token por token. En cada paso la consulta representa lo que ese token busca en el contexto: las consultas vienen del decodificador y los claves y valores provienen de la salida del codificador.

Las tres respuestas comparten el mismo inicio, pero es fundamental notar que dicho inicio (“De forma auto-regresiva, token por token”) es técnicamente incorrecto para describir la atención, correspondiendo en realidad al funcionamiento del decodificador. Al carecer de una representación sólida para este concepto, el modelo sufre una **alucinación**, mezclando términos del dominio.

A partir de ese punto de inseguridad, las tres ejecuciones divergen por completo, evidenciando una distribución de probabilidad muy plana en esa zona de la secuencia. Este resultado corrobora empíricamente las propiedades de la función *softmax* analizadas en el Apéndice B: cuando la puntuación de un token domina claramente el vector $\mathbf{z}_i$, la *softmax* concentra casi toda la probabilidad en él. Cuando ningún token destaca, la distribución se vuelve uniforme y la generación de texto diverge.

Efecto del reentrenamiento

Para verificar la influencia de la aleatoriedad del proceso de entrenamiento, se ha repetido el ajuste fino con el mismo dataset de 480 ejemplos y los mismos hiperparámetros, incluyendo el mismo valor de seed (semilla aleatoria que fija la inicialización de los pesos y el orden de los datos para garantizar la reproducibilidad del experimento). A pesar de ello, los pesos resultantes difieren entre ambos entrenamientos. Esto se debe a que las operaciones de punto flotante en GPU no son estrictamente deterministas: la suma de muchos valores en paralelo puede producir resultados ligeramente diferentes según el orden en que el hardware agrupe las operaciones, ya que la suma en punto flotante no es asociativa. Estas diferencias, del orden de 10^{-7} , se acumulan a lo largo de miles de pasos de entrenamiento y producen conjuntos de pesos distintos.

Al formular las mismas preguntas a ambos modelos, incluso con generación determinista ($T = 0,01$), las respuestas difieren. A diferencia de la variabilidad observada al modificar la temperatura, que proviene de la selección de tokens durante la generación, estas diferencias se originan en los propios pesos del modelo. El siguiente ejemplo ilustra este efecto ante la pregunta “¿Qué pasa si la softmax se satura?” con $T = 0,01$:

Entrenamiento 1: La softmax recibe valores de gran magnitud, produciendo un producto escalar que permite interpretarlos como porcentajes de atención. En esta zona, los gradientes de la softmax son extremadamente pequeños, lo que dificulta el aprendizaje porque las actualizaciones de pesos son minúsculas.

Entrenamiento 2: La softmax recibe valores de gran magnitud que desestabilizan el entrenamiento. En esta zona, los gradientes de las capas cercanas a la salida son muy grandes en la inicialización. Sin embargo, existen técnicas como LoRA que solo modifican un subconjunto pequeño de parámetros, reduciendo el coste computacional.

Ambos modelos identifican correctamente que la saturación implica valores de gran magnitud, pero divergen en la explicación: el primero conecta con los gradientes pequeños (parcialmente correcto), mientras que el segundo mezcla conceptos de inicialización y técnicas de regularización sin relación con la saturación.

También se han realizado las mismas preguntas a ambos modelos con $T = 0,3$. Como resultado, se han encontrado casos en los que ambos producen respuestas correctas pero con formulaciones distintas. Ante la pregunta “¿Qué es una conexión residual?”:

Entrenamiento 1: Es un atajo que suma la entrada de una capa directamente a su salida. En vez de que toda la información pase obligatoriamente por la transformación, parte de ella la rodea. Esto garantiza que el gradiente pueda fluir hacia atrás sin atenuarse.

Entrenamiento 2: Es una modificación arquitectónica que define la salida de una capa como la suma de su entrada con la transformación aplicada: $x^{(l)} = x^{(l-1)} + F(x^{(l-1)})$. En lugar de aprender la transformación completa, la red solo aprende el residuo respecto a la entrada.

Ambas respuestas son correctas, pero mientras la primera opta por una explicación intuitiva, la segunda reproduce la definición formal. Esto indica que los dos entrenamientos han codificado el mismo conocimiento en regiones diferentes del espacio de pesos, produciendo caminos de generación distintos que convergen en el mismo significado. No obstante, este comportamiento es excepcional: en la mayoría de preguntas bien aprendidas, tanto con $T = 0,01$ como con $T = 0,3$ ambos modelos reproducen exactamente la misma secuencia de tokens, lo que refuerza la conclusión de que el modelo memoriza secuencias concretas del dataset en lugar de aprender los conceptos de forma abstracta.

Conclusión

El estudio revela dos fuentes de variabilidad en las respuestas del chatbot. La primera es el proceso de generación, controlado por la temperatura: con $T = 0,01$ las respuestas son deterministas, mientras que con $T = 0,3$ los conceptos inseguros producen respuestas variables. La segunda es el proceso de entrenamiento: incluso con el mismo dataset y los mismos hiperparámetros, la aritmética no determinista de la GPU produce modelos con pesos diferentes que generan respuestas distintas. En ambos casos, la variabilidad se concentra en los conceptos que el modelo no ha aprendido con seguridad, lo que conecta directamente con el análisis de *overfitting* del Capítulo 3.

Apéndice E

Código Fuente

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 plt.rcParams.update({'font.size': 12, 'font.family': 'serif'})
6
7 # Simulamos 10 capas y 15 neuronas por capa
8 num_capas = 10
9 num_pesos = 15
10
11 omega = 0.6
12 gradientes = np.zeros((num_pesos, num_capas))
13
14 # La capa L (índice 9) tiene el gradiente mas alto, la capa 1 (índice 0) el mas bajo
15 for l in range(num_capas):
16     magnitud_base = omega ** (num_capas - 1 - l)
17
18     # Anadimos un poco de ruido aleatorio para que parezca real
19     gradientes[:, l] = magnitud_base * np.abs(np.random.normal(1, 0.2, num_pesos))
20
21
22 plt.figure(figsize=(10, 5))
23 ax = sns.heatmap(gradientes, cmap="magma", cbar_kws={'label': 'Magnitud del Gradiente ($|\partial L / \partial W|$)'})
24
25 ax.set_xlabel('Capa de la Red Neuronal (1 = Entrada, 10 = Salida)')
26 ax.set_ylabel('Índice del Peso (Neurona)')
27 ax.set_xticklabels([f'Capa {i+1}' for i in range(num_capas)], rotation=45)
28
29 plt.tight_layout()
30
31 plt.savefig('vanishing_gradient_heatmap.pdf', format='pdf', dpi=300)
32 plt.show()
```

Listing E.1: Código empleado para generar el mapa de calor de la Figura 1.4.

```
1 import torch
2 import torch.nn.functional as F
3 import math
4
5 # Matriz de entrada X (Embeddings de "Escribe", "un", "email")
6 X = torch.tensor([
7     [2.0, 0.0, 1.0, -1.0], # Escribe
8     [0.0, 1.0, 0.0, 0.0], # un
9     [1.0, 0.0, 2.0, 1.0] # email
```

```

10 ])
11
12 # Matrices de pesos aprendidos (W_Q, W_K, W_V)
13 W_Q = torch.tensor([
14     [1.0, 0.0, 0.0, 0.0],
15     [0.0, 1.0, 0.0, 0.0],
16     [0.0, 0.0, 1.0, 0.0],
17     [0.0, 0.0, 0.0, 1.0]
18 ]) # W_Q es la identidad para simplificar Q = X
19
20 W_K = torch.tensor([
21     [0.0, 0.0, 1.0, 0.0],
22     [0.0, 1.0, 0.0, 0.0],
23     [1.0, 0.0, 0.0, 0.0],
24     [0.0, 0.0, 0.0, -1.0]
25 ])
26
27 W_V = torch.tensor([
28     [5.0, 0.0, 0.0],
29     [0.0, 10.0, 0.0],
30     [0.0, 0.0, 20.0],
31     [0.0, 0.0, 10.0]
32 ]) # Proyecta de d_model=4 a d_v=3
33
34 # Proyecciones
35 Q = torch.matmul(X, W_Q)
36 K = torch.matmul(X, W_K)
37 V = torch.matmul(X, W_V)
38
39 print("Matriz Q:\n", Q)
40 print("Matriz K:\n", K)
41 print("Matriz V:\n", V)
42
43 # Similitudes (Q * K^T)
44 scores = torch.matmul(Q, K.transpose(-2, -1))
45 print("\nMatriz QK^T:\n", scores)
46
47 # Escalado y Softmax
48 d_k = 4
49 scaled_scores = scores / math.sqrt(d_k)
50 A = F.softmax(scaled_scores, dim=-1)
51 print("\nPesos de Atencion: \n ", A)
52
53 # Vector de Contexto Final (Z)
54 Z = torch.matmul(A, V)
55 print("\nMatriz Z (salida): \n ", Z)

```

Listing E.2: Código empleado en el Ejemplo 1 (Atención de Producto Escalar Escalado)

```

1 import torch
2 import torch.nn.functional as F
3 import math
4
5 # Matriz de entrada X (Embeddings de "Escribe", "un", "email")
6 X = torch.tensor([
7     [2.0, 0.0, 1.0, -1.0], # Escribe
8     [0.0, 1.0, 0.0, 0.0], # un
9     [1.0, 0.0, 2.0, 1.0]  # email
10 ])
11
12 # Dimensiones
13 d_model = 4
14 h = 2 # Numero de cabezales
15 d_k = d_v = 2 # Dimension por cabezal (d_model / h)

```

```

16
17 # CABEZAL 1: Q extrae dims 1,3; K extrae dims 3,4
18 W1_Q = torch.tensor([[1.0, 0.0], [0.0, 0.0], [0.0, 1.0], [0.0, 0.0]])
19 W1_K = torch.tensor([[0.0, 0.0], [0.0, 0.0], [1.0, 0.0], [0.0, 1.0]])
20 W1_V = torch.tensor([[1.0, 0.0], [0.0, 1.0], [0.0, 0.0], [0.0, 0.0]])
21
22 Q1 = torch.matmul(X, W1_Q)
23 K1 = torch.matmul(X, W1_K)
24 V1 = torch.matmul(X, W1_V)
25
26
27 print("CABEZAL 1:\n")
28 print("Matriz Q1:\n", Q1)
29 print("Matriz K1:\n", K1)
30 print("Matriz V1:\n", V1)
31
32 scores1 = torch.matmul(Q1, K1.transpose(-2, -1))
33 print("\nMatriz Q1 x K1^T:\n", scores1)
34
35 scaled1 = scores1 / math.sqrt(d_k)
36 print(f"\nMatriz Q1 x K1^T / sqrt({d_k}):\n", scaled1.round(decimals=4))
37
38 A1 = F.softmax(scaled1, dim=-1)
39 print("\nPesos de Atencion A1:\n", A1.round(decimals=4))
40
41 head1 = torch.matmul(A1, V1)
42 print("\nMatriz head_1 (salida cabezal 1):\n", head1.round(decimals=4))
43
44
45 # CABEZAL 2: Q extrae dims 2,4; K extrae dims 1,2
46 W2_Q = torch.tensor([[0.0, 0.0], [1.0, 0.0], [0.0, 0.0], [0.0, 1.0]])
47 W2_K = torch.tensor([[1.0, 0.0], [0.0, 1.0], [0.0, 0.0], [0.0, 0.0]])
48 W2_V = torch.tensor([[0.0, 0.0], [0.0, 0.0], [1.0, 0.0], [0.0, 1.0]])
49
50 Q2 = torch.matmul(X, W2_Q)
51 K2 = torch.matmul(X, W2_K)
52 V2 = torch.matmul(X, W2_V)
53
54 print("CABEZAL 2:\n")
55 print("Matriz Q2:\n", Q2)
56 print("Matriz K2:\n", K2)
57 print("Matriz V2:\n", V2)
58
59 scores2 = torch.matmul(Q2, K2.transpose(-2, -1))
60 print("\nMatriz Q2 x K2^T:\n", scores2)
61
62 scaled2 = scores2 / math.sqrt(d_k)
63 print(f"\nMatriz Q2 x K2^T / sqrt({d_k}):\n", scaled2.round(decimals=4))
64
65 A2 = F.softmax(scaled2, dim=-1)
66 print("\nPesos de Atencion A2:\n", A2.round(decimals=4))
67
68 head2 = torch.matmul(A2, V2)
69 print("\nMatriz head_2 (salida cabezal 2):\n", head2.round(decimals=4))
70
71
72 # Concatenacion y proyeccion final
73 print("CONCATENACION:\n")
74
75 concat = torch.cat([head1, head2], dim=-1)
76 print("\nMatriz Concat(head_1, head_2):\n", concat.round(decimals=4))

```

Listing E.3: Código empleado en el Ejemplo 2 (Atención Multi-Cabezal)

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 import seaborn as sns
4
5 plt.rcParams.update({'font.size': 11, 'font.family': 'serif'})
6
7 d_model = 64
8
9 def calcular_PE(n_posicion, d_modelo):
10     PE = np.zeros((n_posicion, d_modelo))
11     for pos in range(n_posicion):
12         for i in range(0, d_modelo, 2):
13             omega = 1.0 / (10000 ** (i / d_modelo))
14             PE[pos, i] = np.sin(omega * pos)
15             PE[pos, i + 1] = np.cos(omega * pos)
16     return PE
17
18 PE_50 = calcular_PE(50, d_model)
19 PE_1000 = calcular_PE(1000, d_model)
20
21 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6),
22                               gridspec_kw={'width_ratios': [1, 1]})
23
24 # Grafica izquierda: 50 tokens
25 sns.heatmap(PE_50, cmap='RdBu_r', center=0, ax=ax1,
26             cbar=False, xticklabels=False, yticklabels=False)
27 ax1.set_xlabel('Dimension ($i$)', fontsize=13)
28 ax1.set_ylabel('Posicion del token ($pos$)', fontsize=13)
29 ax1.set_title('$n = 50$ posiciones', fontsize=14, fontweight='bold')
30
31 xticks = [0, 16, 32, 48, 63]
32 ax1.set_xticks(xticks)
33 ax1.set_xticklabels(xticks, fontsize=10)
34 yticks_50 = [0, 10, 20, 30, 40, 49]
35 ax1.set_yticks(yticks_50)
36 ax1.set_yticklabels(yticks_50, fontsize=10)
37
38 # Grafica derecha: 1000 tokens
39 im = sns.heatmap(PE_1000, cmap='RdBu_r', center=0, ax=ax2,
40                 cbar=True, cbar_kws={'label': '$PE_{(pos,\\, i)}$'},
41                 xticklabels=False, yticklabels=False)
42 ax2.set_xlabel('Dimension ($i$)', fontsize=13)
43 ax2.set_ylabel('Posicion del token ($pos$)', fontsize=13)
44 ax2.set_title('$n = 1000$ posiciones', fontsize=14, fontweight='bold')
45
46 ax2.set_xticks(xticks)
47 ax2.set_xticklabels(xticks, fontsize=10)
48 yticks_1000 = [0, 200, 400, 600, 800, 999]
49 ax2.set_yticks(yticks_1000)
50 ax2.set_yticklabels(yticks_1000, fontsize=10)
51
52 plt.tight_layout()
53 plt.savefig('positional_encoding_heatmap.pdf', format='pdf', dpi=300)
54 plt.savefig('positional_encoding_heatmap.png', format='png', dpi=300)
55 plt.close()
56
57 print("Figuras generadas correctamente.")

```

Listing E.4: Código empleado para generar el mapa de calor de la Figura 2.4

Apéndice F

Implementación del Chatbot

```
1 # =====
2 # Autor: Victor Sierra Vicen
3 # Archivo: preparar_datos.py
4 # Descripción: Carga el dataset JSON, formatea cada par
5 #               pregunta-respuesta como un prompt para GPT-2 y guarda el
6 #               resultado tokenizado.
7 # Uso: python scripts/preparar_datos.py
8 # =====
9
10 import json
11 import os
12 from transformers import AutoTokenizer
13
14 # =====
15 # CONFIGURACION
16 # =====
17 MODELO_BASE = "DeepESP/gpt2-spanish"
18 DATASET_PATH = os.path.join("data", "dataset_completo.json")
19 OUTPUT_DIR = os.path.join("data", "procesado")
20 MAX_LENGTH = 128 # Longitud máxima en tokens (por ejemplo, se puede cambiar)
21
22 # Tokens especiales para delimitar pregunta y respuesta
23 TOKEN_PREGUNTA = "<pregunta>"
24 TOKEN_RESPUESTA = "<respuesta>"
25 TOKEN_FIN = "<fin>" #Para indicar fin de secuencia
26
27
28 # Carga el dataset json
29 def cargar_dataset(path):
30     with open(path, "r", encoding="utf-8") as f:
31         datos = json.load(f)
32         print(f"Dataset cargado: {len(datos)} pares pregunta-respuesta")
33     return datos
34
35
36 # Convierte una pareja pregunta-respuesta en el formato que el modelo
37 # aprende a generar.
38 # Formato: <pregunta> ... <respuesta> ... <fin>
39 def formatear_prompt(pregunta, respuesta):
40     return f"{TOKEN_PREGUNTA} {pregunta} {TOKEN_RESPUESTA} {respuesta} {TOKEN_FIN}"
41
42
43 # Carga un tokenizer de español estándar y le añade los tokens especiales
44 def preparar_tokenizer(modelo_base):
45     tokenizer = AutoTokenizer.from_pretrained(modelo_base)
46
```

```

47 # GPT-2 no tiene pad_token por defecto, usamos eos_token
48 # Esto es para si una frase es mas corta que la longitud maxima,
49 # se rellena con un token especial de padding
50 tokenizer.pad_token = tokenizer.eos_token
51
52 # Anado los tokens especiales creados
53 tokens_nuevos = [TOKEN_PREGUNTA, TOKEN_RESPUESTA, TOKEN_FIN]
54 num_nuevos = tokenizer.add_special_tokens(
55     {"additional_special_tokens": tokens_nuevos} #Lo paso como diccionario
56 )
57 print(f"Tokens especiales añadidos: {num_nuevos}")
58 print(f"Vocabulario total: {len(tokenizer)} tokens")
59
60 return tokenizer
61
62 # Formatea y tokeniza todos los elementos del database
63 # Marca que tokens pertenecen a la respuesta para que el modelo
64 # solo aprenda a generar esa parte
65 def tokenizar_dataset(datos, tokenizer, max_length):
66     ejemplos_formateados = []
67     ejemplos_descartados = 0
68
69     # ID del token que marca el inicio de la respuesta
70     token_resp_id = tokenizer.convert_tokens_to_ids(TOKEN_RESPUESTA)
71
72     for item in datos: # Un item es un par pregunta-respuesta
73         prompt = formatear_prompt(item["pregunta"], item["respuesta"])
74
75         encoding = tokenizer( #Tokenizo el prompt
76             prompt,
77             max_length=max_length,
78             truncation=True,
79             padding="max_length",
80             return_tensors="pt"
81         )
82
83         input_ids = encoding["input_ids"].squeeze().tolist()
84         attention_mask = encoding["attention_mask"].squeeze().tolist()
85
86         # Verifico longitud, si se ha truncado lo descartamos porque esta
87         # incompleto
88         tokens_sin_padding = sum(attention_mask)
89         if tokens_sin_padding >= max_length:
90             ejemplos_descartados += 1
91             continue
92
93         # Creo labels: -100 en la pregunta y el padding,
94         # solo los tokens de la respuesta contribuyen a la funcion de perdida
95         labels = [-100] * len(input_ids)
96
97         # Encontramos donde empieza la respuesta
98         try:
99             idx_resp = input_ids.index(token_resp_id)
100             # Los tokens desde <respuesta> hasta el final
101             # (excluyendo padding) son los que el modelo aprende
102             for i in range(idx_resp + 1, len(input_ids)):
103                 if attention_mask[i] == 1: #Si no es de padding
104                     labels[i] = input_ids[i]
105             except ValueError:
106                 ejemplos_descartados += 1
107                 continue
108
109         ejemplos_formateados.append({

```



```

109         "input_ids": input_ids,
110         "attention_mask": attention_mask,
111         "labels": labels
112     })
113
114     print(f"Ejemplos procesados: {len(ejemplos_formateados)}")
115     if ejemplos_descartados > 0:
116         print(f"Ejemplos descartados: {ejemplos_descartados}")
117
118     return ejemplos_formateados
119
120 # Guarda los datos tokenizados y el tokenizer modificado
121 def guardar_datos(ejemplos, tokenizer, output_dir):
122
123     os.makedirs(output_dir, exist_ok=True)
124
125     # Guardo los datos tokenizados
126     output_path = os.path.join(output_dir, "train.json")
127     with open(output_path, "w", encoding="utf-8") as f:
128         json.dump(ejemplos, f)
129     print(f"Datos guardados en: {output_path}")
130
131     # Guardo el tokenizer con los tokens especiales
132     tokenizer_path = os.path.join(output_dir, "tokenizer")
133     tokenizer.save_pretrained(tokenizer_path)
134     print(f"Tokenizer guardado en: {tokenizer_path}")
135
136 # Muestra un ejemplo formateado para verificar
137 def mostrar_ejemplo(datos, tokenizer):
138     item = datos[0]
139     prompt = formatear_prompt(item["pregunta"], item["respuesta"])
140     tokens = tokenizer.encode(prompt)
141
142     print("\n" + "=" * 70)
143     print("EJEMPLO DE VERIFICACION")
144     print("=" * 70)
145     print(f"\nPregunta: {item['pregunta']}")
146     print(f"Respuesta: {item['respuesta']}")
147     print(f"\nPrompt formateado:\n{prompt}")
148     print(f"\nNumero de tokens: {len(tokens)}")
149     print(f"Tokens: {tokenizer.convert_ids_to_tokens(tokens[:20])}...")
150     print("=" * 70)
151
152
153 # =====
154 # MAIN
155 # =====
156 if __name__ == "__main__":
157     print("=" * 70)
158     print("PREPARANDO EL DATASET...")
159     print("=" * 70)
160
161     datos = cargar_dataset(DATASET_PATH)
162     tokenizer = preparar_tokenizer(MODELO_BASE)
163     mostrar_ejemplo(datos, tokenizer)
164     ejemplos = tokenizar_dataset(datos, tokenizer, MAX_LENGTH)
165     guardar_datos(ejemplos, tokenizer, OUTPUT_DIR)
166     print("\nPREPARACION COMPLETADA")

```

Listing F.1: Script preparar_datos.py

```

1 # =====
2 # Autor: Victor Sierra Vicen
3 # Archivo: entrenar.py
4 # Descripcion: Fine-tuning de GPT-2 español usando HuggingFace Trainer.
5 #               Compatible con GPU (Google Colab) y CPU.
6 # Uso: python scripts/entrenar.py
7 # =====
8 import json
9 import os
10 import torch
11 from transformers import (
12     AutoModelForCausalLM,
13     AutoTokenizer,
14     TrainingArguments,
15     Trainer,
16     DataCollatorForLanguageModeling
17 )
18 from torch.utils.data import Dataset
19
20 # =====
21 # CONFIGURACION
22 # =====
23 MODELO_BASE = "DeepESP/gpt2-spanish"
24 DATOS_PATH = os.path.join("data", "procesado", "train.json")
25 TOKENIZER_PATH = os.path.join("data", "procesado", "tokenizer")
26 OUTPUT_DIR = "modelo"
27
28 # =====
29 # DATASET
30 # =====
31 class TransformerQADataset(Dataset):
32     def __init__(self, path):
33         with open(path, "r", encoding="utf-8") as f:
34             self.datos = json.load(f)
35             print(f"Dataset: {len(self.datos)} ejemplos", flush=True)
36
37     def __len__(self):
38         return len(self.datos)
39
40     def __getitem__(self, idx):
41         item = self.datos[idx]
42         return {
43             "input_ids": torch.tensor(item["input_ids"], dtype=torch.long),
44             "attention_mask": torch.tensor(item["attention_mask"], dtype=torch.
45             long),
46             "labels": torch.tensor(item["labels"], dtype=torch.long),
47         }
48
49 # =====
50 # ENTRENAMIENTO
51 # =====
52 def entrenar():
53     print("=" * 70, flush=True)
54     print("ENTRENAMIENTO DEL MODELO", flush=True)
55     print("=" * 70, flush=True)
56
57     # Comprueba si hay GPU disponible, si no usa CPU
58     device = "cuda" if torch.cuda.is_available() else "cpu"
59     print(f"Dispositivo: {device}", flush=True)
60
61     # Cargo el tokenizer guardado en preparar_datos.py y el modelo GPT-2
62     print("Cargando tokenizer y modelo...", flush=True)
63     tokenizer = AutoTokenizer.from_pretrained(TOKENIZER_PATH)

```

```

63     modelo = AutoModelForCausalLM.from_pretrained(
64         MODELO_BASE,
65         torch_dtype=torch.float32
66     )
67
68     #Le digo al modelo que haga espacio para los nuevos tokens
69     modelo.resize_token_embeddings(len(tokenizer))
70     num_params = sum(p.numel() for p in modelo.parameters())
71     print(f"Parametros: {num_params:,}", flush=True)
72
73     # Creo una instancia de la clase TransformerQADataset
74     dataset = TransformerQADataset(DATOS_PATH)
75
76     # Data collator: crea labels automaticamente para modelos causales
77     data_collator = DataCollatorForLanguageModeling(
78         tokenizer=tokenizer,
79         mlm=False
80     )
81
82     # Son los argumentos que he ido cambiando a lo largo del entrenamiento
83     training_args = TrainingArguments(
84         output_dir=OUTPUT_DIR,
85         num_train_epochs=50, # Cuantas veces recorre el dataset completo
86         per_device_train_batch_size=4, # Cuantos ejemplos procesa el modelo a la
87         vez
88         learning_rate=3e-5, # Learning rate visto en el documento
89         warmup_steps=50, # Pasos iniciales donde el learning rate aumenta
90         gradualmente
91         weight_decay=0.05, # Regularizacion para evitar el overfitting
92         max_grad_norm=0.5, # Gradient clipping, reescala la norma del
93         gradiente
94         logging_steps=10, # Cada cuantos pasos se imprime la informacion del
95         entrenamiento
96         save_strategy="epoch", # Guarda un checkpoint al final de cada epoca (
97         epoch)
98         save_total_limit=2, # Guarda solo los ultimos dos checkpoints en disco
99         fp16=torch.cuda.is_available(),
100         report_to="none",
101         seed=42, # Fija la semilla para que el entrenamiento sea reproducible
102     )
103
104     # Uso la clase de Hugging Face que se encarga del entrenamiento
105     trainer = Trainer(
106         model=modelo,
107         args=training_args,
108         train_dataset=dataset,
109     )
110
111     # Entreno el modelo
112     print("\nIniciando entrenamiento...", flush=True)
113     print("-" * 70, flush=True)
114     trainer.train()
115
116     # Guardo modelo final ya entrenado
117     print("\nGuardando modelo final...", flush=True)
118     trainer.save_model(OUTPUT_DIR)
119     tokenizer.save_pretrained(OUTPUT_DIR)
120
121     # Guardo el historico de loss para poder generar la grafica
122     log_history = trainer.state.log_history
123     losses = [entry["loss"] for entry in log_history if "loss" in entry]
124     with open(os.path.join(OUTPUT_DIR, "historico_loss.json"), "w") as f:
125         json.dump(losses, f)

```

```

121 print(f"\nModelo guardado en: {OUTPUT_DIR}", flush=True)
122 print(f"Loss final: {losses[-1]:.4f}", flush=True)
123 print("Entrenamiento completado.", flush=True)
124
125
126 if __name__ == "__main__":
127     entrenar()

```

Listing F.2: Script entrenar.py

```

1 # =====
2 # Autor: Victor Sierra Vican
3 # Archivo: chatbot.py
4 # Descripcion: Interfaz de terminal para conversar con el modelo entrenado.
5 # Uso: python scripts/chatbot.py
6 # =====
7
8 import torch
9 from transformers import AutoModelForCausalLM, AutoTokenizer
10
11 # =====
12 # CONFIGURACION
13 # =====
14 MODELO_DIR = "modelo"
15
16 # Parametros de generacion
17 MAX_NEW_TOKENS = 150          # Maximo de tokens a generar
18 TEMPERATURE = 0.3            # Creatividad (0 = determinista, 1 = aleatorio)
19 TOP_K = 30                   # Considera los top-k tokens mas probables
20 TOP_P = 0.85                 # Nucleus sampling
21 REPETITION_PENALTY = 1.5     # Penalizar repeticiones
22
23 # Tokens especiales (deben coincidir con preparar_datos.py)
24 TOKEN_PREGUNTA = "<pregunta>"
25 TOKEN_RESPUESTA = "<respuesta>"
26 TOKEN_FIN = "<fin>"
27
28 # Carga el modelo entrenado y el tokenizer
29 def cargar_modelo():
30     print("Cargando modelo...")
31
32     tokenizer = AutoTokenizer.from_pretrained(MODELO_DIR)
33     modelo = AutoModelForCausalLM.from_pretrained(MODELO_DIR)
34
35     # GPT-2 no tiene pad_token por defecto
36     tokenizer.pad_token = tokenizer.eos_token
37
38     modelo.eval() # Modo inferencia
39
40     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
41     modelo.to(device)
42
43     print(f"Modelo cargado en {device}.")
44     return modelo, tokenizer, device
45
46 # Genera una respuesta a una pregunta. Formatea la entrada como en el
47 # entrenamiento y deja que el modelo complete la respuesta.
48 def generar_respuesta(pregunta, modelo, tokenizer, device):
49
50     # Construye el prompt unicamente de la pregunta
51     prompt = f"{TOKEN_PREGUNTA} {pregunta} {TOKEN_RESPUESTA}"
52
53     # Tokeniza la pregunta
54     inputs = tokenizer(prompt, return_tensors="pt").to(device)

```

```

55
56     # Genera la respuesta
57     with torch.no_grad():
58         outputs = modelo.generate(
59             **inputs,
60             max_new_tokens=MAX_NEW_TOKENS,
61             temperature=TEMPERATURE,
62             top_k=TOP_K,
63             top_p=TOP_P,
64             repetition_penalty=REPETITION_PENALTY,
65             do_sample=True,
66             pad_token_id=tokenizer.eos_token_id,
67             eos_token_id=tokenizer.convert_tokens_to_ids(TOKEN_FIN)
68         )
69
70     # Decodifica solo los tokens generados (sin el prompt)
71     tokens_generados = outputs[0][inputs["input_ids"].shape[1]:]
72     respuesta = tokenizer.decode(tokens_generados, skip_special_tokens=False)
73
74     # Quita el token <fin> y texto posterior a este
75     if TOKEN_FIN in respuesta:
76         respuesta = respuesta[:respuesta.index(TOKEN_FIN)]
77
78     return respuesta.strip()
79
80 # =====
81 # MAIN
82 # =====
83 def main():
84     print("=" * 70)
85     print("CHATBOT SOBRE EL MODELO TRANSFORMER")
86     print("TFG Matematicas - Victor Sierra Vicen")
87     print(70)
88     print()
89
90     modelo, tokenizer, device = cargar_modelo()
91
92     print("\nEscribe tu pregunta sobre el Transformer.")
93     print("Escribe 'salir' para terminar.\n")
94     print("-" * 70)
95
96     while True:
97         pregunta = input("\nTu: ").strip()
98
99         if pregunta.lower() in ["salir", "exit", "quit", "Salir"]:
100             print("\nHasta luego!")
101             break
102
103         if not pregunta:
104             continue
105
106         respuesta = generar_respuesta(pregunta, modelo, tokenizer, device)
107         print(f"\nBot: {respuesta}")
108         print("-" * 70)
109
110 if __name__ == "__main__":
111     main()

```

Listing F.3: Script chatbot.py

El resto de archivos, tanto los descriptivos (README.md, requirements.txt) como los dataset, se encuentran disponibles en GitHub, concretamente en el repositorio Transformer-chatbot: [10].

